

Kennisdelingsplatform

Realisatiedocument

Thomas De Clerck
Simon De Clerck
Bram van Asperen
Studenten Bachelor in de Toegepaste Informatica

Inhoudsopgave

1. INLEIDING	6
2. ANALYSE	7
2.1. Requirements Analysis	7
2.1.1. Inleiding	7
2.1.2. Waarom “Build” boven “Buy”?	8
2.1.3. Use case model	9
2.1.3.1. User Personas	9
2.1.3.2. Use Case Diagram	10
2.1.3.3. Governance & Content Types	12
2.2. Gebruikte tools	13
2.2.1. Inleiding	13
2.2.2. Frontend	13
2.2.3. Backend	15
2.2.4. Authentication	17
2.2.5. Database Management System (DBMS)	19
2.2.6. Versiebeheerplatformen	21
2.2.7. Cloudomgeving	21
2.3. Datamodel en Functionele Scope	25
2.3.1. Entity Relationship Diagram	25
2.3.2. MoSCoW	28
2.4. Prototypes	30
2.5. Conclusie	35
3. REALISATIE	36
3.1. Inleiding	36
3.2. Patronen	36
3.2.1. Gebruikte patronen in de frontend	36
3.2.2. Gebruikte patronen in de backend	37
3.3. Functionaliteiten	38
3.3.1. Navigatiesidebar	38
3.3.2. Algemene layout van de applicatie	41
3.3.3. Tailwind CSS-integratie	42
3.3.4. Font Awesome-integratie	42
3.3.5. Error handling en redirectpagina's in de frontend	43
3.3.6. Topicbeheer	44
3.3.6.1. Detailpagina van topics	44
3.3.6.2. Delete-functionaliteit	45
3.3.6.3. Publicatiefunctie	47
3.3.6.4. Archiveringslogica	48
3.3.7. Interactie en sociale functionaliteiten	49
3.3.7.1. Share-functionaliteit	49
3.3.7.2. Like-functionaliteit	49
3.3.7.3. Rating-functionaliteit	50
3.3.8. Feedbackpagina met EmailJS	51
3.3.9. Tagbeheer CRUD	51

3.3.10. AI-samenvatter met AWS Lambda (TL;DR-functionality)	52
3.3.11. Instelling voor de AI-samenvatter	54
3.3.12. Topic weergaven	55
3.3.13. Userbeheer CRUD met Azure Entra ID	56
3.3.14. Aanvullende bijdragen	57
4. BESLUIT	58
LITERATUURLIJST	59
BIJLAGEN	60

Lijst van figuren

Figuur 1: Use case diagram van het Kennisdelingsplatform	10
Figuur 2: Entity Relationship Diagram van het Kennisdelingsplatform	25
Figuur 3: Indeling van de aggregaat Tag in de backend-mappenstructuur.	37
Figuur 4: Standaardweergave van de navigatiesidebar	38
Figuur 5: Ingeklapte navigatiesidebar met toegang tot gebruikersinstellingen	38
Figuur 6: Mobiele weergave van de navigatiesidebar	39
Figuur 7: Mobiele weergave van de uitgeklapte navigatie	39
Figuur 8: Algemene applicatielay-out met navigatiesidebar en hoofdinhoud	41
Figuur 9: Gebruik van Font Awesome-iconen in de interface	42
Figuur 10: Voorbeeld van een aangepaste foutpagina	43
Figuur 11: Detailpagina van een topic	44
Figuur 12: Topic in het dashboard met delete-actie	45
Figuur 13: Bevestigingsmodal voor het verwijderen van een topic	45
Figuur 14: Bevestiging na succesvolle verwijdering van een topic	46
Figuur 15: Publicatieactie voor een topic in het dashboard	47
Figuur 16: Bevestiging na succesvolle publicatie van een topic	47
Figuur 17: Archiveringsactie voor een topic in het dashboard	48
Figuur 18: Bevestiging na succesvolle archivering van een topic	48
Figuur 19: Share-actie op de detailpagina van een topic	49
Figuur 20: Like-actie op de detailpagina van een topic	49
Figuur 21: Visuele feedback na het liken van een topic	49
Figuur 22: Weergave van het aantal likes in het dashboard	49
Figuur 23: Toegang tot ratingbeheer op de detailpagina van een topic	50
Figuur 24: Delete-actie voor een rating	50
Figuur 25: Bevestiging voor het verwijderen van een rating	50
Figuur 26: Beheerscherm voor tags	51
Figuur 27: AI-samenvatter op de detailpagina van een topic	52
Figuur 28: Gegengereerde AI-samenvatting in de frontend	52
Figuur 29: Communicatiestroom van de AI-samenvatter	53
Figuur 30: AWS Lambda-functie van de AI-samenvatter	53
Figuur 31: Ingeschakelde instelling voor de AI-samenvatter	54
Figuur 32: AI-samenvatter zichtbaar op de detailpagina	54
Figuur 33: Uitgeschakelde instelling voor de AI-samenvatter	54
Figuur 34: AI-samenvatter verborgen op de detailpagina	54
Figuur 35: Aantal weergaven van een topic vóór het openen	55
Figuur 36: Bijgewerkte view counter na het bekijken van een topic	55
Figuur 37: Gebruikersoverzicht binnen de applicatie	56
Figuur 38: Beheer van moderatorrechten voor een gebruiker	56
Figuur 39: Overzicht van de cloudinfrastructuur	60

Lijst van tabellen

Tabel 1: Marktvergelijking van bestaande kennisplatformen	8
Tabel 2: Weighted decision matrix voor frontendtechnologieën	13
Tabel 3: Weighted decision matrix voor backendtechnologieën	15
Tabel 4: Weighted decision matrix voor authenticatietechnologieën	17
Tabel 5: Weighted decision matrix voor databasebeheersystemen	19
Tabel 6: Relaties van het ERD diagram	27
Tabel 7: Pseudocode van de navigatiesidebar	40
Tabel 8: Pseudocode van de redirect logica	43
Tabel 9: Pseudocodefragment van de delete-functionaliteit	46

1. Inleiding

Dit realisatiedocument sluit aan op het eerder opgestelde projectplan voor de stageopdracht rond het kennisdelingsplatform voor Axxes. In dat projectplan werden de context, doelstellingen, business case, planning en teamverdeling van het project beschreven. Daaruit bleek dat Axxes nood heeft aan een centrale en gebruiksvriendelijke toepassing waarin kennis op een gestructureerde manier verzameld, beheerd en gedeeld kan worden.

Binnen Axxes is veel waardevolle kennis aanwezig, onder andere via klantprojecten, interne sessies, opleidingen, documentatie en samenwerking tussen collega's. In de praktijk is die kennis echter vaak verspreid over verschillende tools en kanalen, waardoor het moeilijk wordt om informatie snel terug te vinden of opnieuw te gebruiken. Vanuit die probleemstelling werd de stageopdracht opgezet om een intern kennisdelingsplatform te ontwerpen en te ontwikkelen.

Waar het projectplan vooral beschrijft wat er gerealiseerd moest worden en hoe het project gepland werd, gaat dit realisatiedocument dieper in op de analyse, technische keuzes en effectieve uitvoering van de opdracht. Eerst wordt toegelicht welke onderzoeken en voorbereidingen werden uitgevoerd, zoals de analyse van bestaande oplossingen, de functionele afbakening, de datamodellering, de technologiekeuzes en de projectaanpak. Daarna wordt de realisatie van de applicatie besproken, met aandacht voor de architectuur, de belangrijkste functionaliteiten en de technische principes die tijdens de ontwikkeling werden toegepast.

Het project werd uitgevoerd in teamverband door vier stagiairs, die samen aan de applicatie hebben gewerkt: Thomas De Clerck, Bram van Asperen en Simon De Clerck van Thomas More Hogeschool, en Thijs De Laet van AP Hogeschool. Dit document werd opgesteld door Thomas De Clerck, Bram van Asperen en Simon De Clerck. Thijs De Laet werkte mee aan hetzelfde project, maar stelt vanuit zijn eigen opleiding een apart realisatiedocument op. Het analysehoofdstuk werd door Thomas, Bram en Simon gezamenlijk uitgewerkt, aangezien de voorbereiding, requirements, technologiekeuzes en projectstructuur samen werden onderzocht. Vanaf het hoofdstuk realisatie ligt de nadruk meer op de concrete technische uitwerking en op de individuele bijdragen binnen het project.

Tijdens de stage werden wij begeleid door Michiel Dewilde en Lieven Vetsuypens vanuit Axxes. Zij gaven feedback op de aanpak, technische keuzes en uitwerking van de applicatie. Op die manier kon het project afgestemd worden op de noden van Axxes en verder evolueren tot een oplossing die aansluit bij de praktijk.

Tot slot wordt in dit document teruggeblikt op het eindresultaat en op de mate waarin de doelstellingen uit het projectplan werden gerealiseerd. Daarbij wordt ook stilgestaan bij de belangrijkste inzichten, uitdagingen en mogelijke uitbreidingen voor de toekomst.

2. Analyse

In dit hoofdstuk wordt de voorbereidende analyse van het project toegelicht. Vooraleer met de effectieve realisatie van het kennisdelingsplatform werd gestart, was het belangrijk om eerst duidelijk in kaart te brengen welke noden er binnen Axxes bestaan, welke functionaliteiten relevant zijn en welke technische keuzes het best aansluiten bij de doelstellingen van het project. De analyse vormt daarmee de inhoudelijke en technische basis voor de verdere ontwikkeling van de applicatie.

2.1. Requirements Analysis

2.1.1. Inleiding

In dit hoofdstuk wordt de requirements analysis van het project toegelicht. Eerst worden enkele bestaande tools vergeleken die gebruikt kunnen worden voor kennisdeling en documentatie. Op basis van deze marktvergelijking wordt onderzocht in welke mate standaardoplossingen aansluiten bij de noden van Axxes en waarom er uiteindelijk gekozen werd voor een eigen oplossing.

Daarna wordt de scope van de applicatie verder afgebakend aan de hand van user personas, use cases en een use case diagram. Tot slot worden ook de belangrijkste principes rond governance en content types besproken. Zo wordt duidelijk welke gebruikersnoden centraal staan, welke functionaliteiten de applicatie moet ondersteunen en hoe de inhoud binnen het platform gestructureerd en beheerd kan worden.

2.1.2. Waarom “Build” boven “Buy”?

Uit de marktvergelijking (Zie Tabel 1) blijkt dat bestaande tools zoals SharePoint, Notion en Confluence elk hun sterktes hebben, maar niet volledig aansluiten bij de specifieke noden van Axxes. Daarom werd gekozen voor een eigen oplossing op maat. Deze keuze werd vooral gemaakt omdat het kennisdelingsplatform meerdere soorten kennis moet ondersteunen, de interactie rond kennis zichtbaar moet maken en tegelijk laagdrempelig moet blijven voor consultants.

Tool	Sterktes	Zwaktes	Waarom eigen tool?
SharePoint	Enterprise-grade security; integratie met Microsoft 365.	Informatie raakt verloren in diepe mappenstructuren. Geen native ondersteuning voor code-formatting.	We willen een platform dat <i>leeft</i> , geen archief.
Confluence	Zeer goed voor gestructureerde technische documentatie.	Het onderhouden van pagina’s kost veel tijd. Te rigide voor vluchtige content zoals video-opnames van lunchsessies.	Video, scripts, sessie; niet alleen tekst.
Notion	Mooie UI/UX; zeer flexibel.	Soms is te veel functionaliteit en vrijheid een zwakte.	Wij bouwen governance om wildgroei/chaos te stoppen.
Stack Overflow (Teams)	Perfect voor Q&A en code.	Geen plek voor presentaties of handleidingen, sessies.	We delen meer dan alleen bug-oplossingen; delen ervaringen.

Bron: (Heath, 2025), (Bloomfire, 2025), (Řeřicha, 2025), (Miller, 2024)

Tabel 1: Marktvergelijking van bestaande kennisplatformen

Opmerking: De vergelijking is gebaseerd op publiek beschikbare informatie en op de noden van het project.

Een eerste belangrijke reden is de diversiteit aan content. Binnen Axxes wordt kennis niet op één vaste manier gedeeld. Sommige kennis komt voort uit lunchsessies of presentaties, andere informatie bestaat uit handleidingen, technische documentatie of scripts. Een eigen platform maakt het mogelijk om video, tekst en code samen te brengen in één doorzoekbare interface die afgestemd is op de manier waarop Axxes-medewerkers kennis delen en raadplegen.

Daarnaast is engagement tracking een belangrijke meerwaarde. Door bij te houden welke content gedeeld, bekeken of gewaardeerd wordt, kan beter zichtbaar worden welke kennis leeft binnen de organisatie en waar bepaalde expertise zich bevindt. Dit gaat verder dan enkel documentatie opslaan: het platform helpt ook om expertise binnen Axxes beter vindbaar te maken.

Tot slot was gebruiksgemak een belangrijke factor. Consultants hebben vaak weinig tijd om kennis uitgebreid te documenteren. Daarom moet het proces om content toe te voegen zo laagdrempelig mogelijk zijn. Tegelijk moet de kwaliteit van de inhoud bewaakt kunnen worden door admins of moderators. Door een eigen oplossing te bouwen, kan deze balans tussen weinig drempels voor gebruikers en kwaliteitscontrole door beheerders beter afgestemd worden op de werking van Axxes.

Deze argumenten tonen aan dat een standaardtool onvoldoende flexibel is voor de specifieke combinatie van noden binnen dit project. Een eigen oplossing biedt meer controle over de functionaliteiten, de gebruikerservaring en de manier waarop kennis binnen Axxes beheerd en zichtbaar gemaakt wordt.

2.1.3. Use case model

Om de functionaliteiten van het kennisdelingsplatform af te bakenen, werden de belangrijkste gebruikersrollen en hun noden in kaart gebracht. Op basis daarvan werd bepaald welke acties gebruikers moeten kunnen uitvoeren binnen de applicatie en welke verantwoordelijkheden bij een admin of curator liggen.

2.1.3.1. User Personas

Binnen het platform kunnen drie belangrijke gebruikersprofielen onderscheiden worden: de kennisvrager, de kennisdeler en de admin of curator. Deze profielen helpen om de requirements van de applicatie beter af te stemmen op de verschillende manieren waarop medewerkers het platform zullen gebruiken.

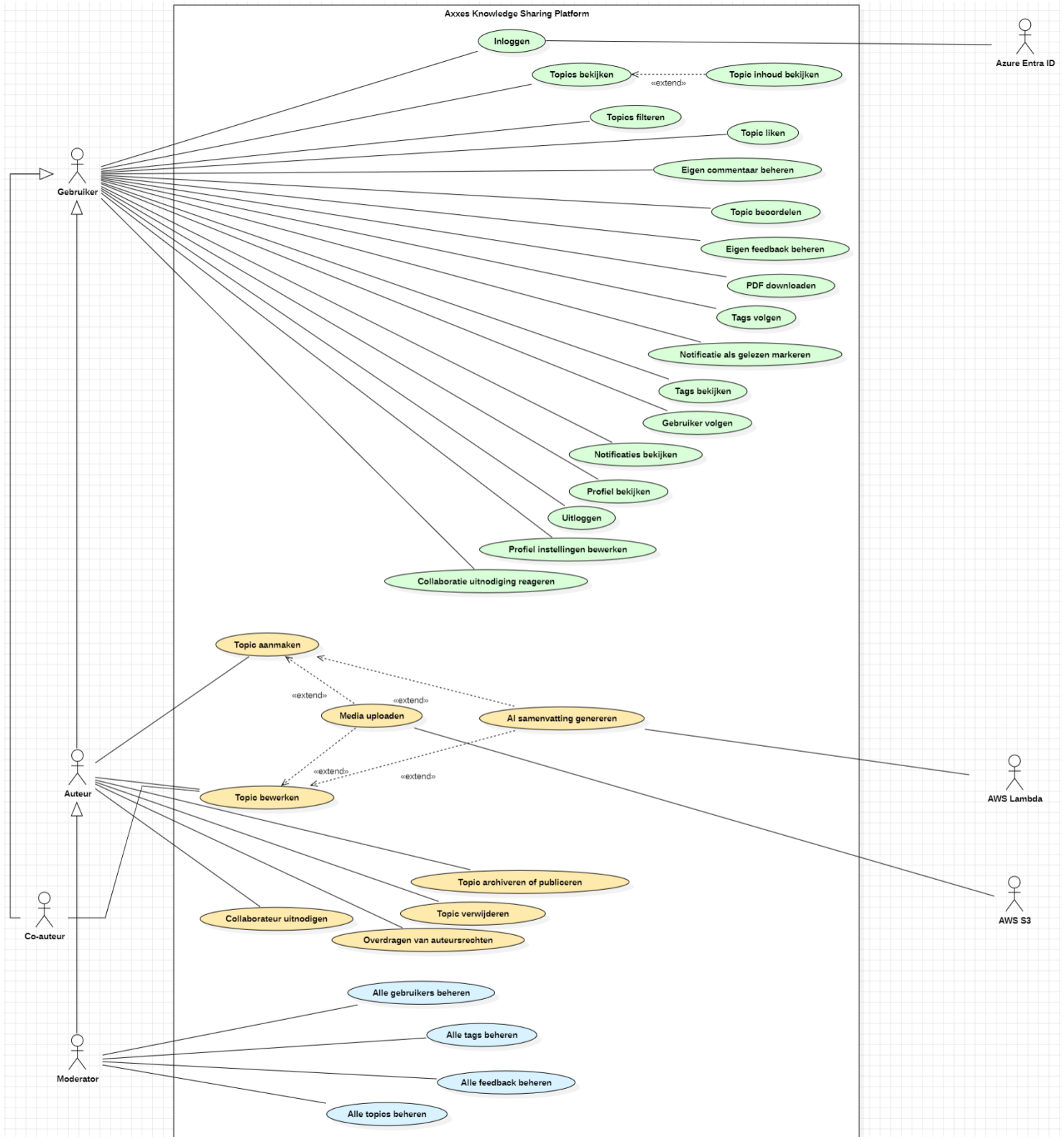
De **kennisvrager** is een medewerker die snel relevante informatie wil terugvinden. Deze gebruiker zoekt antwoorden op concrete vragen, wil op de hoogte blijven van trends binnen Axxes of gebruikt het platform als bron voor zelfstudie. Voor deze persona zijn vooral een duidelijke structuur, een goede zoekfunctie en filters belangrijk.

De **kennisdeler** is een medewerker die eigen kennis, ervaringen of expertise wil delen met collega's. Dit kan bijvoorbeeld gaan over technische oplossingen, handleidingen, scripts, lessons learned of opnames van lunchsessies. Voor deze persona is het belangrijk dat content toevoegen eenvoudig en laagdrempelig blijft, zonder dat dit veel extra administratie vraagt.

De **admin of curator** staat in voor het bewaken van de kwaliteit en structuur van het platform. Deze gebruiker wil overzicht houden over de inhoud, wildgroei of chaos vermijden, verouderde content kunnen archiveren en zicht krijgen op populaire onderwerpen. Voor deze persona zijn beheerfuncties, moderatie en engagementmeting belangrijk.

2.1.3.2. Use Case Diagram

Op basis van deze gebruikersrollen werd een use case diagram opgesteld (zie Figuur 1). Dit diagram geeft visueel weer welke acties de verschillende gebruikers binnen het systeem kunnen uitvoeren. Zo wordt duidelijk welke functionaliteiten beschikbaar zijn voor gewone gebruikers en welke bijkomende verantwoordelijkheden bij de admin of curator liggen.



Figuur 1: Use case diagram van het Kennisdelingsplatform

Gebruiker

De gebruiker is de basisactor binnen het platform. Deze actor stelt een medewerker voor die de applicatie gebruikt om kennis te raadplegen en ermee te interageren. Een gebruiker kan inloggen, topics bekijken, topicinhoud raadplegen, topics filteren, topics liken en topics beoordelen. Daarnaast kan de gebruiker eigen commentaar beheren, feedback geven, tags bekijken of volgen, notificaties raadplegen en profielinstellingen aanpassen.

Deze functionaliteiten vormen de algemene basiswerking van het platform. Ze zorgen ervoor dat gebruikers kennis kunnen terugvinden, opvolgen en beoordelen. Ook interacties zoals likes, ratings, collaboratie en comments dragen bij aan de betrokkenheid van gebruikers binnen het platform.

Auteur

De auteur is een uitbreiding van de gewone gebruiker. Deze actor kan naast de algemene gebruikersfunctionaliteiten ook zelf topics aanmaken en beheren. Een auteur kan topics aanmaken, bestaande topics bewerken, topics verwijderen en topics archiveren of publiceren.

Daarnaast kan een auteur media uploaden bij een topic en een AI-samenvatting laten genereren. Deze functionaliteiten ondersteunen het aanmaken van rijkere en beter bruikbare content. De auteur speelt daardoor een centrale rol in het actief delen en onderhouden van kennis binnen het platform.

Co-auteur

De co-auteur is betrokken bij samenwerking rond topics. Deze actor kan worden uitgenodigd om mee te werken aan een topic en kan op een collaboratie-uitnodiging reageren. Hierdoor wordt het mogelijk om kennis niet alleen individueel, maar ook samen met andere gebruikers uit te werken.

De co-auteur bouwt voort op de rol van auteur en ondersteunt vooral de samenwerking rond contentcreatie binnen het platform.

Moderator

De moderator beschikt over extra beheermogelijkheden binnen het platform. Deze actor kan gebruikers beheren, tags beheren, feedback beheren en topics beheren. Deze functionaliteiten zijn belangrijk om de kwaliteit, structuur en veiligheid van de gedeelde inhoud te bewaken.

Door deze beheermogelijkheden kan een moderator ingrijpen wanneer inhoud foutief, ongepast of verouderd is. Daarnaast kan de moderator ook helpen om de structuur van het platform overzichtelijk te houden, bijvoorbeeld door tags en feedback op te volgen.

Externe systemen

Naast de interne gebruikersrollen bevat het use case diagram ook externe systemen. Azure Entra ID wordt gebruikt voor de authenticatie van gebruikers. Hierdoor kunnen gebruikers aanmelden met hun bedrijfsaccount en wordt de toegang tot het platform gekoppeld aan de bestaande Microsoft-omgeving van Axxes.

AWS Lambda wordt gebruikt binnen de AI-samenvatter. Deze component verwerkt de aanvraag om een samenvatting te genereren en vormt de koppeling met de achterliggende AI-functionaliteit. AWS S3 wordt gebruikt voor het verwerken of opslaan van bestanden en media die aan topics gekoppeld worden.

Relaties tussen use cases

In het diagram komen ook extend-relaties voor. Deze geven aan dat bepaalde functionaliteiten optioneel of aanvullend zijn binnen een andere use case. Zo kan bij het aanmaken of bewerken van een topic bijkomende functionaliteit worden uitgevoerd, zoals media uploaden of een AI-samenvatting genereren.

Ook bij het bekijken van topics kan aanvullende functionaliteit aanwezig zijn, zoals het bekijken van de volledige topicinhoud. Deze relaties tonen hoe bepaalde onderdelen van de applicatie logisch met elkaar verbonden zijn en hoe sommige acties voortbouwen op een hoofdfunctionaliteit.

2.1.3.3. Governance & Content Types

Naast de functionele use cases werd ook nagedacht over de manier waarop content binnen het platform beheerd en gestructureerd moet worden. Een kennisplatform is pas waardevol wanneer informatie niet alleen toegevoegd kan worden, maar ook relevant, betrouwbaar en overzichtelijk blijft. Daarom werden binnen de analyse ook principes rond governance en content types meegenomen.

Governance speelt hierbij een belangrijke rol. Om de kwaliteit en betrouwbaarheid van de gedeelde kennis te bewaken, moet het mogelijk zijn om bepaalde content te markeren als Axxes Verified. Dit label geeft aan dat de inhoud als waardevol of betrouwbaar wordt beschouwd en helpt gebruikers om sneller kwalitatieve informatie te herkennen. Op die manier wordt niet enkel ingezet op het verzamelen van kennis, maar ook op het bewaken van de kwaliteit ervan.

Ook admins of curators krijgen hierin een belangrijke beheerrol. Zij moeten kunnen ingrijpen wanneer informatie verouderd, foutief of niet meer relevant is. Daarom moeten zij content kunnen archiveren, verwijderen of beheren waar nodig. Zo wordt vermeden dat het platform uitgroeit tot een ongestructureerde verzameling van informatie en blijft de gedeelde kennis bruikbaar op lange termijn.

Tot slot moet het platform verschillende content types ondersteunen. Kennis binnen Axxes komt namelijk niet altijd in dezelfde vorm voor. Sommige informatie bestaat uit tekstuele handleidingen of documentatie, terwijl andere kennis gedeeld wordt via video's, codevoorbeelden of externe links. Door meerdere content types te ondersteunen, wordt het platform bruikbaar voor verschillende soorten kennisdeling.

Voorbeelden van ondersteunde content types zijn:

- **Video:** Opnames of links naar interne sessies, lunchsessies of presentaties.
- **Tekst:** Handleidingen, documentatie, stappenplannen of lessons learned.
- **Code snippets:** Syntaxblokken voor technische voorbeelden die snel hergebruikt kunnen worden.
- **Externe links:** Verwijzingen naar nuttige externe bronnen, documentatie of tools.
- **Bijlagen:** Bestanden die extra context of ondersteunend materiaal bevatten.
- ...

2.2. Gebruikte tools

2.2.1. Inleiding

In dit hoofdstuk bespreken we de tools die binnen dit project in aanmerking kwamen. Eerst lichten we per tool kort toe wat de tool/technologie doet en waarom deze relevant is voor ons project. Daarna vergelijken we de verschillende opties aan de hand van een *decision matrix*, waarbij we rekening houden met vooraf bepaalde criteria en wegingsfactoren.

Op basis van deze vergelijking maken we een onderbouwde keuze voor de tool die het best aansluit bij de noden en doelstellingen van het project. Tot slot formuleren we een conclusie waarin we onze eindkeuze en de bijhorende motivatie duidelijk toelichten.

2.2.2. Frontend

Angular, React en Vue.js werden vergeleken op basis van criteria zoals leercurve, community, prestaties, schaalbaarheid, onderhoudbaarheid en beschikbare libraries (zie Tabel 2).

Geëvalueerde technologieën:

- Angular (gekozen)
- React
- Vue.js

Angular: Is een frontend framework ontwikkeld door Google voor het bouwen van dynamische webapplicaties. Het biedt een volledige structuur met ingebouwde oplossingen voor routing, formulieren en state management, waardoor het geschikt is voor grotere en complexere projecten.

React: Is een JavaScript library ontwikkeld door Meta voor het bouwen van gebruikersinterfaces, vooral op basis van herbruikbare componenten. Het is zeer populair door zijn flexibiliteit, grote community en sterke prestaties in interactieve webapplicaties.

Vue.js: Is een frontend framework dat bekendstaat om zijn eenvoud en gebruiksvriendelijkheid. Het combineert een lage instapdrempel met krachtige functionaliteiten, waardoor het geschikt is voor zowel kleine als grotere webprojecten.

	Angular	React	Vue.js	Weighting
Leercurve	2	3	5	1x
Community & documentatie	4	5	3	6x
Prestatie (Rendering snelheid)	3	4	5	5x
Schaalbaarheid	5	4	3	4x
Onderhoudbaarheid	5	3	4	3x
Libraries	5	5	3	2x
Totaalscore	86	88	78	

Tabel 2: Weighted decision matrix voor frontendtechnologieën

Opmerking: De scores zijn gebaseerd op documentatie, online vergelijkingen en de noden van het project.

Onderstaande toelichting verduidelijkt waarom deze scores werden toegekend.

- **Leercurve:** **Vue.js** wint hier omdat het zeer toegankelijk is voor beginners (Ahmad, 2025), (Shah, 2026). **Angular** heeft de steilste leercurve vanwege de complexe architectuur en het verplichte gebruik van TypeScript en RxJS.
- **Community & documentatie:** **React** is de absolute marktleider met de grootste community en de meeste tutorials/oplossingen op StackOverflow (Ahmad, 2025), (Tech Formation, 2025). **Angular** volgt sterk door de corporate support van Google.
- **Prestatie (Rendering snelheid):** (Mika & Vasylenko, 2026) legt uit dat React's Virtual DOM zeer efficiënt is, maar dat **Vue.js** vaak nog snellere rendering-resultaten behaalt in benchmarks omdat het lichter is. Angular wordt door (Tech Formation, 2025) als krachtiger maar "zwaarder" omschreven.
- **Schaalbaarheid & Onderhoudbaarheid:** **Angular** blinkt hier uit. Omdat het een "opinionated" framework is, dwingt het een vaste structuur af (Tech Formation, 2025). Dit maakt het makkelijker om met grote teams aan enorme enterprise-projecten te werken zonder dat de code een rommeltje wordt.
- **Libraries:** **React** heeft voor elk probleem wel tien externe libraries. **Angular** heeft daarentegen bijna alles al "ingebouwd", wat volgens (Shah, 2026) een enorme vorm van rijkdom en stabiliteit biedt.

Conclusie

Met een score van 88 tegenover 86 is het verschil tussen React en Angular nu minimaal. De keuze hangt nu af van de voorkeur van het team. Of de community van React of de strikte structuur van Angular.

Wij hebben gekozen voor de meer gestructureerde enterprise-solution wat in teamverband met een structuur dus makkelijker zou zijn. Dus kiezen wij voor Angular.

2.2.3. Backend

De backendtechnologieën werden met elkaar vergeleken op basis van criteria zoals leercurve, community, authenticatie, schaalbaarheid, onderhoudbaarheid en testing (zie Tabel 3).

Geëvalueerde technologieën:

- Java REST API (gekozen)
- .NET Web API
- Python FastAPI

Java REST API: Is een robuuste, objectgeoriënteerde taal die door vele enterprise-omgevingen gebruikt wordt. Met frameworks zoals Spring Boot biedt het een goede set aan tools voor beveiliging en schaalbaarheid.

.NET Web API: Is een krachtig framework van Microsoft, soms ook vergelijkbaar met Java. Het staat bekend om zijn uitstekende prestaties, sterke integratie met cloud-omgevingen en uitgebreide tooling voor grote applicaties.

Python FastAPI: Is een modern, high-performance framework gebaseerd op Python. Het is extreem snel te ontwikkelen en is zeer goed in asynchrone taken, maar mist soms de architectuur van de grotere frameworks.

	Java REST API	.NET Web API	Python FastAPI	Weighting
Leercurve	3	4	5	1x
Community & documentatie	5	5	4	6x
Authenticatie & autorisatie	5	5	3	2x
Schaalbaarheid	5	5	4	4x
Onderhoudbaarheid	5	5	4	5x
Testing & debugging	5	5	4	3x
Totaalscore	103	104	83	

Tabel 3: Weighted decision matrix voor backendtechnologieën

Opmerking: De scores zijn gebaseerd op documentatie, online vergelijkingen en de noden van het project.

Onderstaande toelichting verduidelijkt waarom deze scores werden toegekend.

- **Leercurve:** **FastAPI** is hier de duidelijke winnaar dankzij de eenvoudigheid van Python en de automatische documentatie (Swagger). **Java** heeft de laagste score vanwege de complexe "boilerplate" code en de diepe kennis die nodig is van het Spring-ecosysteem (Subramanian, 2026).
- **Community & documentatie:** Zowel **Java** als **.NET** scoren maximaal. Ze bestaan al lang en hebben een antwoord op elk denkbaar probleem op Stack Overflow. FastAPI groeit snel, maar mist nog de enorme bibliotheek aan enterprise-oplossingen die de andere twee hebben.
- **Authenticatie & autorisatie:** **Java (Spring Security)** en **.NET Identity** zijn de gouden standaard voor complexe beveiliging. Ze bieden "out-of-the-box" ondersteuning voor OAuth2, JWT en LDAP. FastAPI biedt de bouwstenen, maar vereist meer handmatige configuratie voor complexe enterprise-scenario's.
- **Schaalbaarheid & Onderhoudbaarheid:** **Java** en **.NET** dwingen een strikte architectuur af (bijv. Dependency Injection). Dit voorkomt dat grote projecten veranderen in code die niet meer een structuur heeft. Volgens (Subramanian, 2026) is dit essentieel voor applicaties die jarenlang mee moeten gaan.
- **Testing & debugging:** De tools voor Java (JUnit, Mockito) en .NET (xUnit) is uiterst goed. De IDE-ondersteuning voor het debuggen van complexe problemen is in deze talen beter dan die van Python.

Conclusie

Met een score van **104 tegenover 103** ligt het verschil tussen .NET en Java laag. .NET wint op papier nipt vanwege een iets vriendelijkere leercurve voor developers.

Voor ons moet de keuze voor de backend in lijn liggen met de frontend strategie en de beschikbare expertise. Omdat wij hebben gekozen voor **Angular** (een framework dat sterk leunt op Java-achtige principes zoals TypeScript, Dependency Injection en een strikte mappenstructuur), sluit de **Java REST API** hier goed op aan. Daarom kiezen wij voor Java REST API.

2.2.4. Authentication

De authenticatietechnologieën werden vergeleken op basis van criteria zoals leercurve, documentatie, beveiligingsopties, schaalbaarheid, integratie en kosten (zie Tabel 4).

Geëvalueerde technologieën:

- Azure Entra ID (gekozen)
- AWS Cognito
- Auth0

Azure Entra ID: Een enterprise-identity service van Microsoft die gespecialiseerd is in identiteits- en toegangsbeheer. Het biedt geavanceerde beveiligingsfuncties zoals Conditional Access.

AWS Cognito: De identity service van AWS voor het toevoegen van gebruikersbeheer aan web- en mobiele apps. Het is diep geïntegreerd met andere AWS-services, wat configuratie in de cloud vergemakkelijkt.

Auth0: Een flexibel Identity-as-a-Service (IDaaS) platform dat bekendstaat om een goede developer experience en eenvoudige integratie via diverse SDK's, maar vaak duurder uitvalt bij opschaling.

	Azure Entra ID	AWS Cognito	Auth0	Weighting
Leercurve	3	3	5	1x
Community & documentatie	5	4	5	2x
Beveiligingsopties	5	4	5	6x
Schaalbaarheid	5	5	4	3x
Integratie (AWS/Java)	5	5	4	4x
Kosten	4	5	2	5x
Totaalscore	98	95	83	

Tabel 4: Weighted decision matrix voor authenticatietechnologieën

Opmerking: De scores zijn gebaseerd op documentatie, online vergelijkingen en de noden van het project.

Onderstaande toelichting verduidelijkt waarom deze scores werden toegekend.

- **Leercurve:** Uit vergelijkingen op (G2.com, n.d.) blijkt dat Auth0 veruit de hoogste score haalt op het gebied van "Ease of Setup" en "Ease of Use". Azure Entra ID en AWS Cognito scoren lager omdat de administratieve consoles complexer zijn en een diepere kennis van het Microsoft- of AWS-ecosysteem vragen.
- **Community & documentatie:** Zowel Microsoft als Auth0 scoren maximaal. Azure Entra ID heeft een enorme informatiebank van Microsoft, terwijl Auth0 ontwikkelaarsgerichte documentatie en SDK-ondersteuning heeft (G2.com, n.d.).
- **Beveiligingsopties:** Volgens (Gartner, Inc. and/or its affiliates, n.d.) wordt Microsoft (Entra ID) gepositioneerd als een absolute "Leader". De diepte van de beveiligingsfuncties, zoals *Identity Protection* en risicogebaseerde toegangscontrole, wordt door Gartner hoger ingeschat dan de standaard functionaliteiten van AWS Cognito.
- **Schaalbaarheid:** In de Gartner Peer Insights scoren zowel Azure als AWS zeer hoog op schaalbaarheid en betrouwbaarheid voor miljoenen gebruikers. Auth0 krijgt een iets lagere score vanwege de soms beperkende rate-limits.
- **Integratie (AWS/Java):** Hoewel AWS Cognito de native keuze is voor AWS, krijgt Azure Entra ID eveneens dezelfde score. Dit komt doordat Entra ID de industriestandaard voor OIDC/SAML-protocollen volgt, wat integratie met de gekozen Java Spring Security backend garandeert.
- **Kosten:** Uit gebruikersreviews op G2 en prijsanalyses blijkt dat Auth0 de duurste optie is zodra enterprise-features (zoals SAML of MFA) nodig zijn. AWS Cognito is het meest voordelig voor developers. Azure Entra ID is echter uiterst kosteneffectief voor organisaties die reeds Microsoft 365-licenties bezitten, aangezien de basisfunctionaliteit vaak al inbegrepen is (Gartner, Inc. and/or its affiliates, n.d.) (G2.com, n.d.).

Conclusie

Azure Entra ID de hoogste score van **98**. Hoewel Auth0 uitblinkt in gebruiksvriendelijkheid en AWS Cognito de meest logische keuze lijkt binnen een AWS-cloudomgeving, biedt Azure Entra ID de beste keuze voor een enterprise-omgeving.

2.2.5. Database Management System (DBMS)

De databasebeheersystemen werden vergeleken op basis van criteria zoals data-integriteit, community, enterprise features, schaalbaarheid, cloud-ondersteuning en kosten (zie Tabel 5).

Geëvalueerde technologieën:

- Oracle DB
- PostgreSQL
- MySQL (gekozen)

Oracle DB: Een krachtig, enterprise-grade relationeel databasebeheersysteem (RDBMS). Het staat bekend om zijn extreme betrouwbaarheid, geavanceerde beveiligingsfuncties en het vermogen om zeer complexe transacties en enorme hoeveelheden data te verwerken.

PostgreSQL: Een geavanceerd open-source RDBMS dat dichtbij aansluit bij de standaarden van Oracle. Het biedt zeer krachtige functies voor complexe queries en datatypes, en is een goed alternatief in cloud-omgevingen vanwege de flexibiliteit en kosteneffectiviteit.

MySQL: Wereldwijd een van de meest gebruikte open-source databases. Het is betrouwbaar en snel voor standaard webapplicaties, maar mist soms de diepgaande analytische en enterprise-functies die Oracle en PostgreSQL bieden.

	Oracle DB	PostgreSQL	MySQL	Weighting
Data-integriteit & ACID	5	5	4	6x
Community & documentatie	5	4	5	1x
Enterprise features	5	4	3	4x
Schaalbaarheid	5	4	4	3x
Cloud-ondersteuning (AWS)	5	5	5	2x
Kosten	2	5	5	5x
Totaalscore	90	97	88	

Tabel 5: Weighted decision matrix voor databasebeheersystemen

Opmerking: De scores zijn gebaseerd op documentatie, online vergelijkingen en de noden van het project.

Onderstaande toelichting verduidelijkt waarom deze scores werden toegekend.

- **Data-integriteit & ACID:** Oracle en PostgreSQL scoren maximaal. Volgens de vergelijking op (El khirani, 2024) bieden beide systemen de meest strikte naleving van ACID-eigenschappen, wat nodig is voor het voorkomen van data-corruptie bij complexe transacties. MySQL scoort iets lager, hoewel de InnoDB-engine ACID-compliant is, biedt het iets minder geavanceerde constraints en triggers vergeleken met de andere twee (Roach, 2024).
- **Community & documentatie:** Oracle scoort maximaal door de uitgebreide 24/7 enterprise-support. MySQL heeft de grootste community-ondersteuning wereldwijd, terwijl PostgreSQL een zeer sterke professionele achtergrond heeft.
- **Enterprise features:** Oracle is de absolute leider op het gebied van geavanceerde beveiliging, partitioning en *high availability*. (Gartner, Inc. and/or its affiliates, 2025) positioneert Oracle consistent in het "Leaders" kwadrant voor Cloud Database Management Systems vanwege deze diepgaande functionaliteiten. MySQL is eenvoudiger, het is geoptimaliseerd voor webapplicaties maar mist de analytische functies die PostgreSQL aanbiedt (Roach, 2024).
- **Schaalbaarheid:** Oracle biedt zowel verticale als horizontale schaalbaarheid voor enorme datasets. PostgreSQL en MySQL scoren goed in cloud-omgevingen, maar Oracle wordt door (Redgate Software, 2026) nog steeds aangewezen als de standaard voor de meest veeleisende enterprise-workloads.
- **Cloud-ondersteuning (AWS):** Alle drie de systemen scoren maximaal. AWS biedt via **Amazon RDS** volledige ondersteuning voor alle drie, inclusief automatische backups en schaalbaarheid. De integratie met Java is voor alle drie de opties goed en gestandaardiseerd.
- **Kosten:** Dit is het grootste nadeel van Oracle. De licentiekosten zijn hoger dan die van PostgreSQL en MySQL, die als open-source software geen licentiekosten hebben (El khirani, 2024).

Conclusie

De matrix laat zien dat PostgreSQL met een score van **94** mathematisch het best scoort, voornamelijk door de sterke balans tussen functionaliteit en kost. Oracle Database volgt met **90** en presteert technisch zeer sterk, maar brengt een aanzienlijk hogere licentie en beheerkost met zich mee.

Uiteindelijk werd MySQL gekozen omdat het binnen de context van deze stageopdracht de meest haalbare optie was. Ten eerste is MySQL in vergelijking met de alternatieven kostenefficiënt (geen zware licentiekosten), wat belangrijk is voor een intern platform met een beperkt projectbudget. Ten tweede was MySQL de meest gekende technologie binnen het team, waardoor de implementatie sneller en met minder risico kon gebeuren.

Tot slot sloot deze keuze aan bij de voorkeur en ervaring van de stagebegeleiders, wat de technische ondersteuning en opvolging tijdens de ontwikkeling vereenvoudigde. Daardoor bood MySQL de beste combinatie van lage kost, snelle opstart en beperkte projectrisico's voor dit project.

2.2.6. Versiebeheerplatformen

Binnen deze categorie van versiebeheerplatformen hebben we GitHub, GitLab en Bitbucket bekeken. Hoewel alle drie geschikte opties zijn voor codebeheer en samenwerking, werd in dit project gekozen voor Bitbucket. De reden hiervoor is dat Axxes Bitbucket als standaardplatform gebruikt binnen hun werking. Om aan te sluiten bij de bestaande werkwijze van het stagebedrijf en de samenwerking zo vlot mogelijk te laten verlopen, volgen wij deze keuze en gebruiken we Bitbucket binnen dit project.

2.2.7. Cloudomgeving

Voor de cloudfrastructuur van dit project wordt gebruikgemaakt van een combinatie van Amazon Web Services (AWS) en Microsoft Azure. AWS wordt ingezet voor de hosting, deployment, opslag, database en infrastructuur van de applicatie. Azure wordt gebruikt voor de authenticatie via Azure Entra ID.

Deze keuze werd gemaakt omdat de applicatie op die manier later eenvoudiger geïntegreerd kan worden met de bestaande identiteitsstructuur van Axxes. Gebruikers kunnen zich aanmelden met hun bestaande bedrijfsaccount, waardoor er geen aparte gebruikersaccounts binnen de applicatie zelf aangemaakt of beheerd moeten worden.

De algemene cloudfarchitectuur van de applicatie wordt weergegeven in Figuur 39 in de bijlage. Dit schema toont hoe de verschillende AWS- en Azure-componenten met elkaar verbonden zijn binnen de applicatieflow.

Omgeving

Binnen de cloudfrastructuur zijn er twee afzonderlijke omgevingen voorzien: een stagingomgeving en een productieomgeving.

De stagingomgeving wordt gebruikt om nieuwe code die klaar is voor validatie online te testen. Hierdoor kan gecontroleerd worden hoe de applicatie zich gedraagt in een cloudomgeving, met echte infrastructuur en databronnen. Dit is belangrijk omdat een applicatie lokaal anders kan reageren dan wanneer ze effectief online draait. De stagingomgeving dient dus als tussenstap tussen lokale ontwikkeling en productie.

De productieomgeving bevat de stabiele versie van de applicatie die bedoeld is voor eindgebruikers. Enkel gevalideerde builds worden naar deze omgeving gebracht. Op die manier wordt vermeden dat ongeteste wijzigingen rechtstreeks beschikbaar worden voor gebruikers.

Deze scheiding tussen staging en productie zorgt ervoor dat nieuwe functionaliteiten eerst gecontroleerd kunnen worden voordat ze in gebruik genomen worden in de productieomgeving.

Backend deployment

Zoals weergegeven in Figuur 39 draait de backend op Amazon ECS. ECS wordt gebruikt om de backend als container uit te voeren binnen AWS. Hierdoor kan de backend op een gecontroleerde en schaalbare manier worden beheerd.

Voor elke backenddeployment wordt eerst een Docker-image aangemaakt. Deze image bevat de volledige backendapplicatie en alle nodige configuratie om de applicatie correct te kunnen uitvoeren. Nadat de image gebouwd is, wordt deze geüpload naar Amazon ECR, de container registry van AWS.

Amazon ECS gebruikt vervolgens de juiste image uit ECR om een nieuwe versie van de backend te starten. Dit gebeurt via een nieuwe task revision. Dankzij deze revision kan duidelijk opgevolgd worden welke versie van de backend op een bepaald moment actief is.

Voor elke backendbuild wordt een buildtag aangemaakt. Deze tag bestaat uit een combinatie van de build, de Bitbucket-commit of UUID en de applicatieversie. Een voorbeeld hiervan is:

build-commit-version

Bijvoorbeeld:

build-abc123-1.0.0

Deze buildtag maakt het mogelijk om exact te identificeren welke versie van de backend gebouwd en gedeployed werd.

Voor de productieomgeving wordt er geen nieuwe image gebouwd. In plaats daarvan wordt een bestaande image gepromoot. Dit betekent dat dezelfde image die eerder al werd aangemaakt en getest in de stagingflow ook gebruikt wordt voor productie. Hierdoor wordt vermeden dat er verschillen ontstaan tussen de versie die getest werd en de versie die uiteindelijk in productie terechtkomt.

Frontend deployment

Zoals weergegeven in Figuur 39 wordt de frontend gehost via Amazon S3. Voor zowel de stagingomgeving als de productieomgeving is er een aparte S3-bucket voorzien. Hierdoor blijven de frontendbestanden van beide omgevingen gescheiden.

De Angular-applicatie wordt tijdens de deployment gebouwd naar statische bestanden. Deze bestanden worden vervolgens naar de juiste S3-bucket gekopieerd. Vanuit deze bucket kan de applicatie online beschikbaar worden gemaakt.

In tegenstelling tot de backend wordt de frontend niet gepromoot tussen staging en productie. Door tijdsbeperkingen werd ervoor gekozen om de frontend per omgeving afzonderlijk te deployen naar de juiste S3-bucket.

Om de frontend efficiënter en veiliger beschikbaar te maken voor gebruikers, wordt gebruikgemaakt van Amazon CloudFront. CloudFront werkt als content delivery network en plaatst de statische frontendbestanden dicht bij de gebruiker. Hierdoor kan de applicatie sneller geladen worden. Daarnaast vormt CloudFront ook een extra laag tussen de gebruiker en de S3-bucket.

CI/CD via Bitbucket Pipelines

Voor het automatisch bouwen, testen en deployen van de applicatie wordt gebruikgemaakt van Bitbucket Pipelines. Deze pipeline zorgt ervoor dat het deploymentproces zoveel mogelijk geautomatiseerd verloopt.

De pipeline start met het uitvoeren van de nodige tests voor zowel de backend als de frontend. Pas wanneer deze tests succesvol zijn, worden de volgende stappen uitgevoerd. Dit voorkomt dat code die niet door de testfase geraakt automatisch naar AWS wordt gedeployed.

Na de testfase worden de deploymentstappen voor de frontend en backend parallel uitgevoerd.

Voor de backend houdt dit in dat er een Docker-image wordt gebouwd. Deze image wordt vervolgens gepusht naar Amazon ECR. Daarna kan Amazon ECS deze image gebruiken om de backend te deployen met de correcte revision.

Voor de frontend wordt de Angular-applicatie gebouwd. De gegenereerde statische bestanden worden daarna gekopieerd naar de juiste S3-bucket, afhankelijk van de omgeving waarvoor de deployment wordt uitgevoerd.

Voor een productie-deployment moet expliciet een buildtag worden meegegeven. De pipeline controleert vervolgens of deze buildtag effectief bestaat in Amazon ECR. Alleen wanneer de opgegeven image beschikbaar is, kan de deployment naar productie doorgaan. Dit voorkomt dat er per ongeluk een foutieve of niet-bestaande versie naar productie wordt gebracht.

Database

Zoals weergegeven in Figuur 39 communiceert de backend met een MySQL-database die gehost wordt via Amazon RDS. Amazon RDS is een beheerde relationele databaseservice, waardoor AWS een deel van het operationele beheer van de database-infrastructuur overneemt.

Op die manier blijft de data laag geïntegreerd binnen dezelfde cloudinfrastructuur als de backend.

Het gebruik van Amazon RDS heeft als voordeel dat de database niet volledig manueel beheerd moet worden op een eigen server. Zaken zoals infrastructuurbeheer, beschikbaarheid en onderhoud worden gedeeltelijk door AWS ondersteund. Hierdoor kan de focus meer liggen op de applicatielogica en minder op het beheer van de onderliggende database-infrastructuur.

Opslag van afbeeldingen

Voor het opslaan van afbeeldingen wordt een aparte Amazon S3-bucket gebruikt. Deze bucket wordt gedeeld door zowel de stagingomgeving als de productieomgeving.

Zoals weergegeven in Figuur 39 communiceert de backend met deze S3-bucket om afbeeldingen of andere mediabestanden op te slaan en op te halen. Hierdoor worden bestanden niet rechtstreeks op de backendserver bewaard.

Wanneer gebruikers of de applicatie afbeeldingen nodig hebben, worden deze centraal opgeslagen en opgehaald uit deze gedeelde bucket. Door hiervoor S3 te gebruiken, kunnen bestanden op een schaalbare manier worden opgeslagen.

Deze aanpak zorgt ervoor dat de backend stateless kan blijven. De backend hoeft de bestanden zelf niet lokaal te bewaren, maar kan verwijzen naar de objecten die in S3 opgeslagen zijn.

Netwerk, distributie en toegangsbeheer

Naast de applicatiecomponenten zelf maakt de cloudomgeving ook gebruik van verschillende AWS-services voor netwerkbeheer, domeinconfiguratie, distributie en toegangsbeheer.

Amazon VPC vormt de netwerklaag van de infrastructuur. Binnen de VPC worden de verschillende AWS-resources logisch georganiseerd en afgeschermd. Dit maakt het mogelijk om bijvoorbeeld de backend, database en andere interne services op een gecontroleerde manier met elkaar te laten communiceren.

Amazon Route 53 wordt gebruikt voor DNS-beheer. Hiermee worden domeinnamen gekoppeld aan de juiste AWS-resources. Zo kan de applicatie bereikbaar worden gemaakt via duidelijke en herkenbare URL's in plaats van via technische AWS-endpoints.

Amazon CloudFront wordt gebruikt voor de distributie van de frontend. De Angular-applicatie staat opgeslagen in Amazon S3, maar wordt via CloudFront beschikbaar gemaakt aan de gebruiker. Hierdoor worden statische bestanden sneller geleverd en wordt de S3-bucket niet rechtstreeks als publieke toegangspoort gebruikt.

AWS IAM wordt gebruikt voor toegangsbeheer binnen AWS. Via IAM worden rechten en rollen toegekend aan gebruikers, services en pipelines. Hierdoor krijgen componenten enkel toegang tot de resources die ze effectief nodig hebben. Dit principe wordt ook wel het principle of least privilege genoemd.

Een voorbeeld hiervan is dat de Bitbucket Pipeline enkel rechten nodig heeft om images naar ECR te pushen, bestanden naar S3 te kopiëren of een ECS-service te updaten. De pipeline krijgt dus geen volledige toegang tot alle AWS-services, maar enkel tot wat nodig is voor de deployment.

Door deze services te gebruiken, wordt de cloudomgeving veiliger, beter beheersbaar en geschikter voor een professionele productieomgeving.

Aanvullende AWS-services

Naast de hoofdcomponenten van de infrastructuur wordt er ook gebruikgemaakt van AWS Lambda en Amazon Bedrock.

AWS Lambda wordt ingezet voor serverless functionaliteiten. Dit betekent dat bepaalde stukken functionaliteit uitgevoerd kunnen worden zonder dat hiervoor een aparte server of container permanent actief moet zijn. Lambda is vooral nuttig voor taken die slechts op bepaalde momenten uitgevoerd moeten worden.

Amazon Bedrock wordt gebruikt voor AI-functionaliteiten binnen de applicatie. Via Bedrock kunnen generatieve AI-modellen geïntegreerd worden zonder dat deze modellen volledig zelf gehost of beheerd moeten worden. Hierdoor kan de applicatie gebruikmaken van AI-mogelijkheden terwijl het beheer van de onderliggende modellen grotendeels bij AWS blijft.

Authenticatie via Azure Entra ID

Voor authenticatie wordt gebruikgemaakt van Azure Entra ID. Dit betekent dat gebruikers zich niet aanmelden via een apart gebruikerssysteem binnen de applicatie, maar via de bestaande Microsoft-identiteitsomgeving.

De authenticatiestroom wordt ook weergegeven in Figuur 39. Wanneer een gebruiker zich wil aanmelden, wordt hij vanuit de Angular-frontend doorgestuurd naar Azure Entra ID. Daar voert de gebruiker zijn aanmeldgegevens in. Na een succesvolle authenticatie ontvangt de frontend een token.

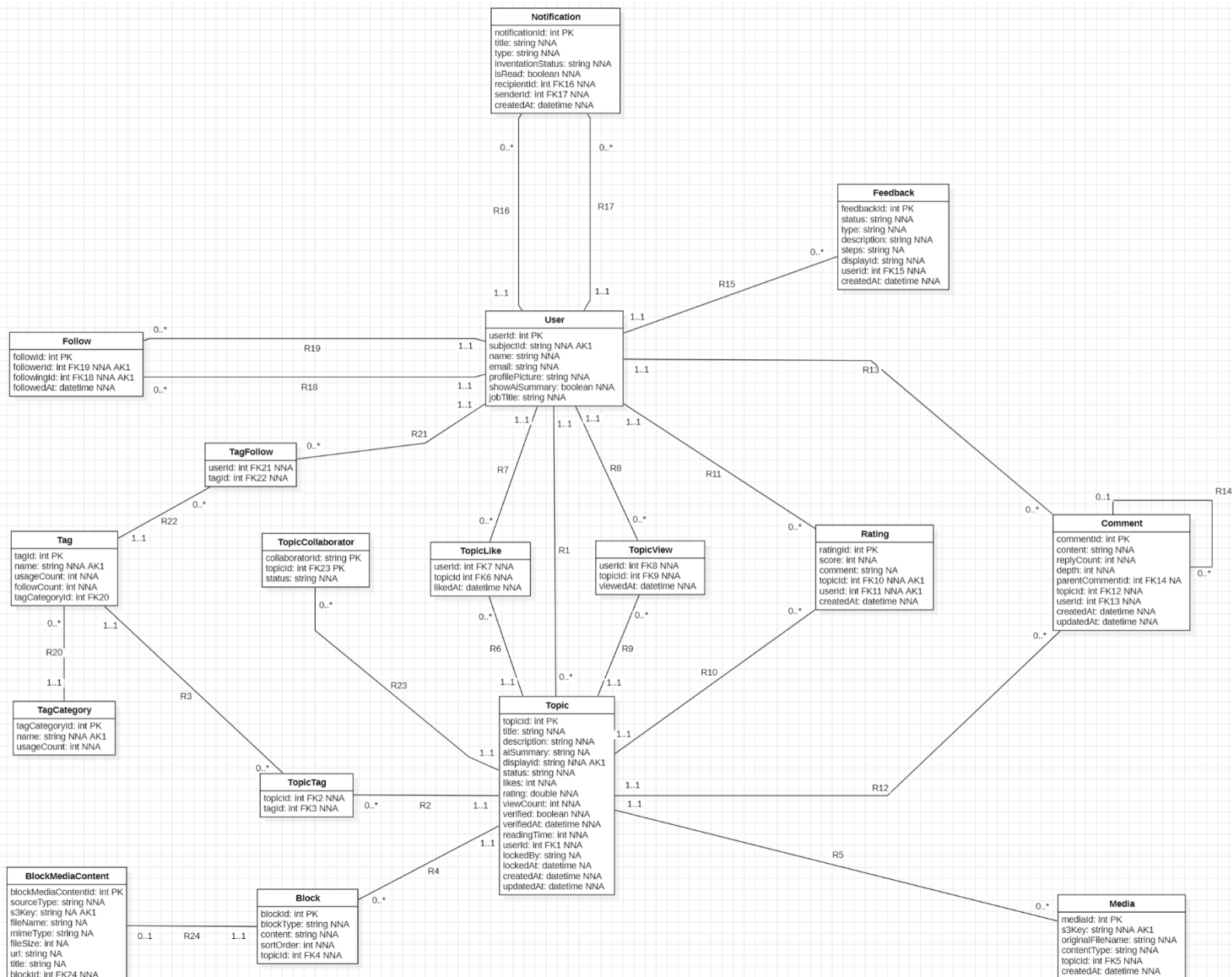
Dit token wordt vervolgens gebruikt om beveiligde aanvragen naar de backend te sturen. De Java Spring Boot-backend valideert het token voordat toegang wordt gegeven tot beveiligde functionaliteiten of data.

Door Azure Entra ID te gebruiken, kan de applicatie later eenvoudiger aansluiten op de bestaande gebruikersstructuur van Axxes. Dit maakt het beheer van gebruikers en rechten eenvoudiger en voorkomt dat medewerkers aparte accounts nodig hebben voor deze applicatie.

2.3. Datamodel en Functionele Scope

In dit hoofdstuk beschrijven we welke data noodzakelijk en relevant is voor de applicatie en hoe deze gestructureerd wordt. Om dit duidelijk in kaart te brengen, maken we gebruik van een Entity Relationship Diagram (zie Figuur 2), waarin de entiteiten, hun attributen en onderlinge relaties worden weergegeven. Daarnaast wordt met behulp van een MoSCoW-analyse verduidelijkt welke functionaliteiten als essentieel, wenselijk of optioneel beschouwd worden binnen het systeem.

2.3.1. Entity Relationship Diagram



Figuur 2: Entity Relationship Diagram van het Kennisdelingsplatform

User

Deze entiteit bevat alle gebruikers van de applicatie. Hierin staan gegevens zoals naam, e-mailadres, profielfoto, jobtitel en persoonlijke instellingen zoals het tonen van AI-samenvattingen.

Topic

Deze entiteit bevat de kennisartikels of onderwerpen die gebruikers aanmaken. Een topic heeft onder andere een titel, beschrijving, status, auteur, aantal likes, rating, views en eventueel een AI-samenvatting.

TopicCollaborator

Deze entiteit stelt een collaborator-koppeling voor. Ze koppelt een topic aan de subject id van de gebruiker die collaborator is. De combinatie van topicId en collaboratorId vormt samen de primary key, zodat dezelfde collaborator niet dubbel aan hetzelfde topic gekoppeld kan worden.

Tag

Deze entiteit bevat labels die aan topics gekoppeld kunnen worden. Tags worden gebruikt om topics te categoriseren, bijvoorbeeld op basis van technologie of league.

TagCategory

Deze entiteit houdt bij welke tags bij welke categorieën aangewezen zijn.

TopicTag

Deze entiteit is een koppeltabel tussen topics en tags. Ze zorgt ervoor dat één topic meerdere tags kan hebben en dat één tag aan meerdere topics gekoppeld kan worden.

TagFollow

Deze entiteit houdt bij welke tags door welke gebruikers gevolgd worden. Hierdoor kan de applicatie interesses van gebruikers bijhouden op basis van tags.

Block

Deze entiteit bevat de inhoudsblokken van een topic. Een topic kan bestaan uit meerdere blokken, zoals tekst, code, afbeeldingen, video's, audio of bestanden.

Media

Deze entiteit bevat informatie over geüploade mediabestanden. Hierin staan bijvoorbeeld de S3-key, originele bestandsnaam, content type en het topic waaraan het bestand gekoppeld is.

TopicLike

Deze entiteit houdt bij welke gebruikers welke topics geliket hebben. Hierdoor kan gecontroleerd worden dat een gebruiker een topic maar één keer liket.

TopicView

Deze entiteit houdt bij welke gebruikers welke topics bekeken hebben. Dit wordt gebruikt om het aantal views van een topic correct bij te houden.

Rating

Deze entiteit bevat beoordelingen die gebruikers aan topics geven. Een rating bestaat uit een score, eventueel een commentaar, de gebruiker die de rating gaf en het topic waarop de rating hoort.

Comment

Deze entiteit bevat reacties op topics. Een comment hoort bij een topic en een gebruiker, en kan ook een antwoord zijn op een andere comment.

Follow

Deze entiteit houdt bij welke gebruikers andere gebruikers volgen. Dit maakt het mogelijk om een volgsysteem te hebben waarbij gebruikers elkaar kunnen volgen.

Notification

Deze entiteit bevat meldingen voor gebruikers. Een notificatie heeft een ontvanger, een verzender en informatie over het topic waarop de melding betrekking heeft.

Feedback

Deze entiteit bevat feedback die gebruikers geven over de applicatie. Dit kan bijvoorbeeld een bug, UI-opmerking, feature request of andere feedback zijn. Feedback heeft ook een status om de opvolging ervan bij te houden.

BlockMediaContent

Deze entiteit bevat extra media-informatie voor een block. Dit kan bijvoorbeeld informatie zijn over een geüpload bestand of een externe URL.

Relatie	Uitleg
R1	Als gebruiker kan ik meerdere topics aanmaken en elk topic heeft één auteur.
R2 & R3	Een topic kan meerdere tags hebben en één tag kan aan meerdere topics gekoppeld worden.
R4	Een topic kan meerdere inhoudsblokken bevatten en elk blok hoort bij één topic.
R5	Een topic kan meerdere mediabestanden hebben en elk mediabestand kan aan één topic gekoppeld zijn.
R6 & R7	Als gebruiker kan ik meerdere topics liken en een topic kan door meerdere gebruikers geliket worden.
R8 & R9	Als gebruiker kan ik meerdere topics bekijken en een topic kan door meerdere gebruikers bekeken worden.
R10 & R11	Als gebruiker kan ik ratings geven aan topics en een topic kan meerdere ratings ontvangen.
R12 & R13	Als gebruiker kan ik meerdere comments plaatsen en elke comment heeft één auteur.
R14	Een comment kan meerdere antwoorden hebben en een antwoord hoort bij maximaal één parent comment.
R15	Als gebruiker kan ik meerdere feedback-items aanmaken en elk feedback-item hoort bij één gebruiker.
R16	Als gebruiker kan ik meerdere notificaties ontvangen en elke notificatie heeft één ontvanger.
R17	Als gebruiker kan ik meerdere notificaties veroorzaken of verzenden en elke notificatie heeft één verzender.
R18 & R19	Als gebruiker kan ik meerdere gebruikers volgen en een gebruiker kan door meerdere gebruikers gevolgd worden.
R20	Een tag category kan meerdere tags bevatten en elke tag hoort bij één tag category.
R21 & R22	Als gebruiker kan ik meerdere tags volgen en een tag kan door meerdere gebruikers gevolgd worden.
R23	Een topic kan meerdere collaborator-records hebben en elk collaborator-record hoort bij één topic. De collaborator wordt bijgehouden via collaboratorId, dat de subject id van de gebruiker bevat.
R24	Een block kan maximaal één block media content hebben en elke block media content hoort bij één block.

Tabel 6: Relaties van het ERD diagram

2.3.2. MoSCoW

Must have

Authenticatie en toegang

De applicatie moet gebruikers toelaten om in te loggen via Azure Entra ID. Op die manier wordt de toegang gekoppeld aan de bestaande Microsoft-omgeving van Axxes.

Topics aanmaken, bekijken en beheren

Gebruikers moeten topics kunnen aanmaken en bestaande topics kunnen bekijken. Daarnaast moeten zij hun eigen topics kunnen bewerken of verwijderen wanneer dat nodig is.

Interactie via comments en likes

Om kennisdeling interactiever te maken, moeten gebruikers kunnen reageren op topics. Zij moeten ook hun eigen comments kunnen bewerken en verwijderen. Daarnaast moeten topics geliket kunnen worden, zodat gebruikers waardering kunnen tonen voor relevante inhoud.

Topics filteren en doorzoeken

Topics moeten gefilterd kunnen worden en doorzoekbaar zijn op basis van titel of technologie. Hierdoor kunnen gebruikers sneller relevante kennis terugvinden.

Structuur en metadata van topics

Topics moeten gekoppeld kunnen worden aan een league, technologie en contenttype. Daarnaast moeten topics meerdere attachments kunnen bevatten, zodat verschillende soorten informatie toegevoegd kunnen worden.

Beheer door administrator

Een administrator moet gebruikers, topics en comments kunnen beheren. Dit is nodig om de kwaliteit en veiligheid van de gedeelde inhoud te bewaken.

Should have

Kwaliteitslabel voor betrouwbare content

Een administrator kan content labelen als Axxes Verified. Dit label helpt gebruikers om betrouwbare of waardevolle informatie sneller te herkennen.

Inzicht in gebruik en engagement

Populaire topics worden visueel gemarkeerd. Daarnaast wordt engagement tracking voorzien, bijvoorbeeld door het aantal views per topic bij te houden.

Persoonlijk overzicht

Gebruikers krijgen een profielpagina met een overzicht van hun aangemaakte topics. Dit maakt het eenvoudiger om eigen bijdragen terug te vinden en op te volgen.

Could have

Uitgebreidere dashboards en statistieken

Een dashboard met statistieken per gebruiker kan extra inzicht geven in activiteit en betrokkenheid binnen het platform.

Trending en erkenning

Een trending topics-sectie kan populaire inhoud extra zichtbaar maken. Daarnaast kan een badge- of erkenningssysteem gebruikers motiveren om actief kennis te delen.

Geavanceerde zoek- en bewaarfuncties

Geavanceerde zoekfilters kunnen het eenvoudiger maken om specifieke inhoud terug te vinden. Naast likes kan ook een bookmark-functionaliteit voorzien worden, zodat gebruikers topics kunnen bewaren voor later.

Personalisatie

Een white/dark mode kan worden toegevoegd om de gebruikerservaring verder te personaliseren.

AI-ondersteuning

AI-gegenereerde samenvattingen van topics kunnen gebruikers helpen om sneller de kern van langere topics te begrijpen.

Won't have

Publieke toegang

Externe publieke toegang wordt niet voorzien, omdat het platform bedoeld is als interne tool voor Axxes-medewerkers.

Realtime chat

Realtime chatfunctionaliteit wordt niet opgenomen in de scope. De focus ligt op gestructureerde kennisdeling via topics en comments.

Externe kennisplatformen

Integraties met externe kennisplatformen, zoals de StackOverflow API, worden niet voorzien binnen deze versie van de applicatie.

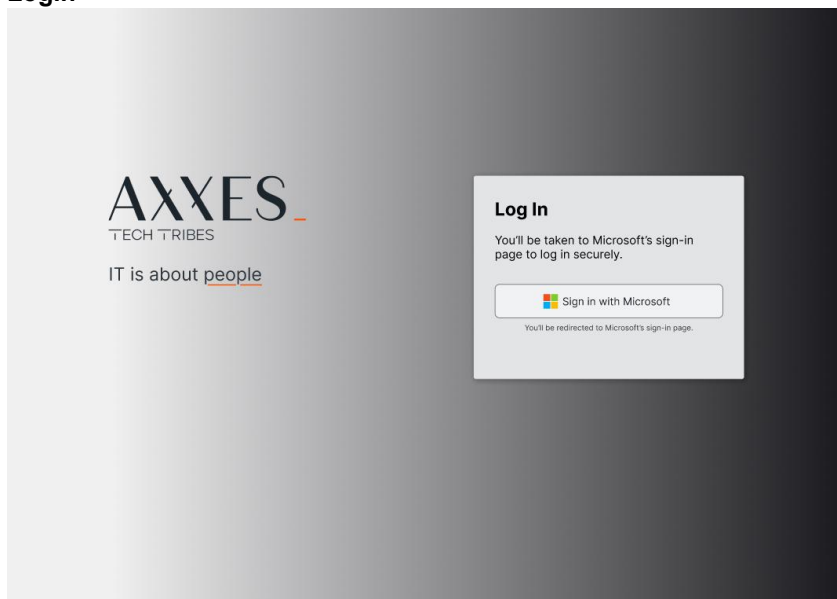
2.4. Prototypes

In dit hoofdstuk presenteren we de schermen die we in Figma hebben uitgewerkt. Deze ontwerpen geven een visueel overzicht van de gebruikersflow, de belangrijkste functionaliteiten en de opbouw van de interface.

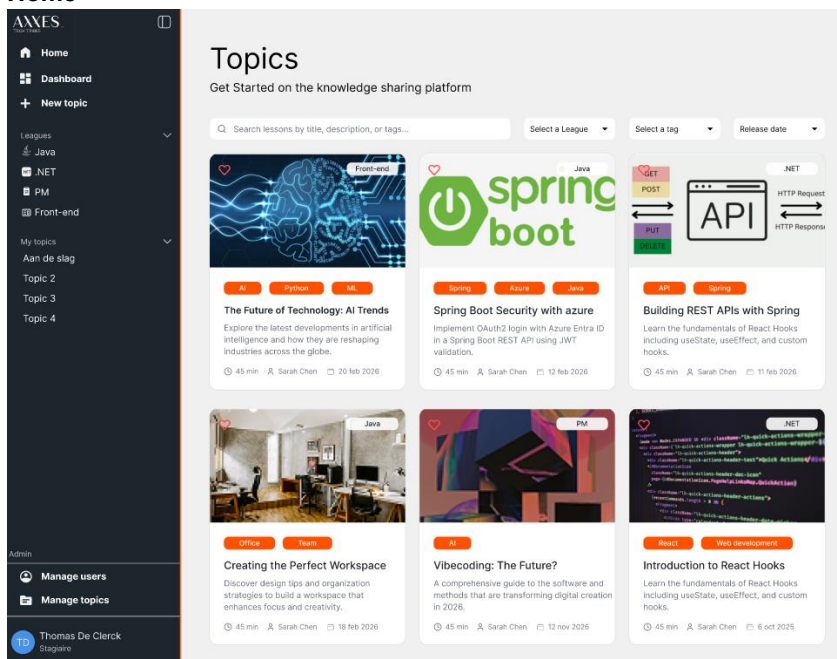
De weergegeven Figma-prototypes stellen de initiële ontwerpen van de applicatie voor. De uiteindelijke realisatie wijkt hiervan op meerdere punten af door verdere technische en functionele evoluties tijdens het project.

Figma link: <https://www.figma.com/proto/rjjPgYDGVJjNjwoaJhgzik/Kennisdelingsplatform?node-id=51-128&p=f&t=YQvCMn1h9VSy8SN6-1&scaling=contain&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=51%3A128>

a) Login



b) Home



c) Profile

AXXES

Home

Dashboard

New topic

Leagues

Java

.NET

PM

Front-end

My topics

Aan de slag

Topic 2

Topic 3

Topic 4

Admin

Manage users

Manage topics

TD

Thomas De Clerck

Stagiaire

Profile information

Update your account's profile information and email address.

Profile icon

TD

Select a new photo

Name

Thomas De Clerck

Email

thomas.de.clerck@example.com

Save

Browser sessions

Manage and log out your active sessions on other browsers and devices.

If necessary, you may log out of all of your other browser sessions across all of your devices. If you feel your account has been compromised, you should also update your password.

Log out other browser session

d) Dashboard

AXXES

Home

Dashboard

New topic

Leagues

Java

.NET

PM

Front-end

My topics

Aan de slag

Topic 2

Topic 3

Topic 4

Admin

Manage users

Manage topics

TD

Thomas De Clerck

Stagiaire

Dashboard

Manage and track your published content

Total Views

1356

+15.3% from last month

Total Favourites

296

+10.7% from last month

Axxes verified

10 topics

Topics written

35

My topics

The Future of Technology: AI and Machine Learning Trends

Explore the latest developments in artificial intelligence and how they are reshaping industries across the globe.

Feb 20, 2026 12,543 428

Creating the Perfect Workspace for Maximum Productivity

Discover design tips and organization strategies to build a workspace that enhances focus and creativity.

Feb 18, 2026 8,762 315

Vibecoding: The Future of Developers

A comprehensive guide to the software and methods that are transforming digital creation in 2026.

Feb 15, 2026 15,234 892

Introduction to React Hooks

Learn the fundamentals of React Hooks including useState, useEffect, and custom hooks.

Feb 12, 2026 9,876 567

< Previous

1

2

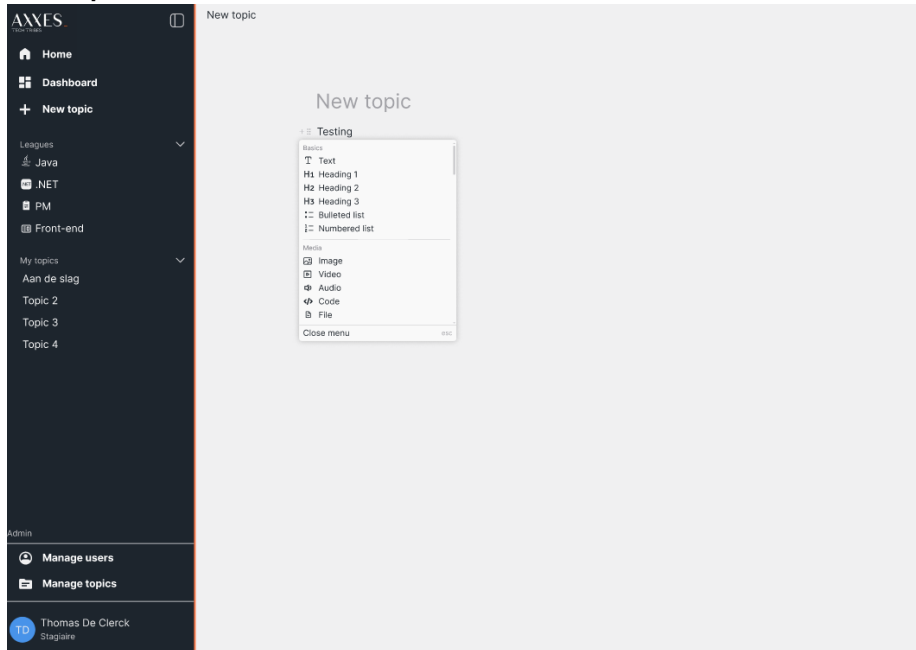
3

Next >

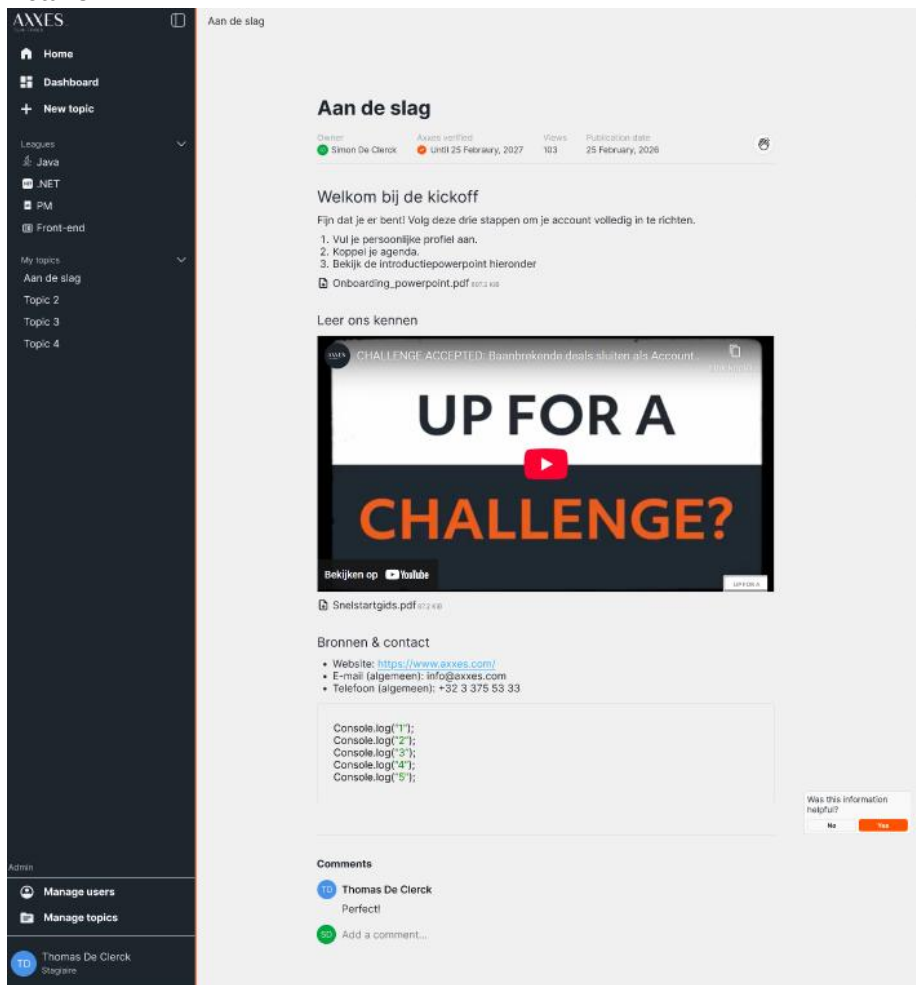
Stage – Realisatiedocument

31

e) New topic



f) Details



g) User CRUD

AXXES

[Home](#)
[Dashboard](#)
[+ New topic](#)

Leagues

[Java](#)
[.NET](#)
[PM](#)
[Front-end](#)

My topics

[Aan de slag](#)
[Topic 2](#)
[Topic 3](#)
[Topic 4](#)

Admin

[Manage users](#)
[Manage topics](#)

TD

Thomas De Clerck

Stagiaire

User management

Manage your users

[+ Add Employee](#)

Name	Email	League	Seniority	Status	Actions
Sarah Johnson	sarah.j@axxes.com	Java	Senior	Active	Edit Delete
Michael Chen	michael.c@axxes.com	.NET	Medior	Active	Edit Delete
Emma Wilson	emma.w@axxes.com	Front-end	Junior	Inactive	Edit Delete
Jeffrey De Clerck	jeffrey.dc@axxes.com	Front-end	Medior	Inactive	Edit Delete
Bob van Asperen	bob.va@axxes.com	Java	Junior	Active	Edit Delete

[< Previous](#)
[1](#)
[2](#)
[3](#)
[Next >](#)

h) Topic CRUD

AXXES

[Home](#)
[Dashboard](#)
[+ New topic](#)

Leagues

[Java](#)
[.NET](#)
[PM](#)
[Front-end](#)

My topics

[Aan de slag](#)
[Topic 2](#)
[Topic 3](#)
[Topic 4](#)

Admin

[Manage users](#)
[Manage topics](#)

TD

Thomas De Clerck

Stagiaire

Topic management

Manage topics

[+ Add Topic](#)

Title	Views	Likes	Actions
Aan de slag	20	5	Edit Delete
Topic 2	#	#	Edit Delete
Topic 3	#	#	Edit Delete
Topic 4	#	#	Edit Delete
Topic 5	#	#	Edit Delete

[< Previous](#)
[1](#)
[2](#)
[3](#)
[Next >](#)

i) Editing

AXXES

Home

Dashboard

New topic

Leagues

Java

.NET

PM

Front-end

My topics

Aan de slag

Topic 2

Topic 3

Topic 4

Admin

Manage users

Manage topics

Thomas De Clerck

Stagiaire

Aan de slag

Aan de slag

Welkom bij de kickoff

Fijn dat je er bent! Volg deze drie stappen om je account volledig in te richten.

1. Vul je persoonlijke profiel aan.

2. Koppel je agenda.

3. Bekijk de introductiepowerpoint hieronder

Onboarding_powerpoint.pdf 897.2 KB

Leer ons kennen

CHALLENGE ACCEPTED: Baanbrekende deals sluiten als Account...

UP FOR A CHALLENGE?

Bekijken op YouTube

Snelstartgids.pdf 872 KB

Bronnen & contact

Website: <https://www.axxes.com/>

E-mail (algemeen): info@axxes.com

Telefoon (algemeen): +32 3 375 53 33

```
Console.log("1");
Console.log("2");
Console.log("3");
Console.log("4");
Console.log("5");
```

Cancel

Republish

AXXES

Home

Dashboard

New topic

Leagues

Java

.NET

PM

Front-end

My topics

Aan de slag

Topic 2

Topic 3

Topic 4

Admin

Manage users

Manage topics

Thomas De Clerck

Stagiaire

Publish topic

Fill in the details and assign tags/league to publish your topic

Topic Title *

Aan de slag

Owner

Simon De Clerck

Short Description

Assign Tags *

+ Add Tags

No tags assigned yet. Click "Add Tags" to select relevant tags or league.

Cancel

Publish

2.5. Conclusie

Uit de analyse blijkt duidelijk dat er binnen Axxes nood is aan een centraal en gebruiksvriendelijk kennisdelingsplatform. De requirementanalyse, de user personas en de marktvergelijking tonen aan dat bestaande tools wel bepaalde functionaliteiten aanbieden, maar niet volledig aansluiten bij de specifieke verwachtingen en werking van de organisatie. Daarom werd bewust gekozen voor een oplossing op maat, waarin verschillende content types gecombineerd kunnen worden en waarin tegelijk voldoende aandacht is voor structuur, beheer en toegankelijkheid.

Daarnaast heeft de analyse geholpen om de al bestaande technische keuzes verder te controleren en te onderbouwen. Omdat Thijs al drie weken vroeger aan het project was begonnen, waren bij onze opstart al enkele technologieën naar voren geschoven op basis van zijn eerdere analyse. Wij hebben deze keuzes echter niet zomaar overgenomen, maar opnieuw onderzocht en vergeleken om na te gaan of ze daadwerkelijk de beste match vormden met de doelstellingen van het project, de context van Axxes en de samenwerking binnen ons team. Sommige keuzes, zoals Azure Entra ID, waren bovendien al bepaald door Axxes zelf. In die gevallen lag de nadruk van onze analyse vooral op het bevestigen en beargumenteren van die beslissing. De uiteindelijke technologiecombinatie vormt zo een onderbouwd en samenhangend geheel voor de verdere uitwerking van de applicatie.

Ook de diagrammen, prototypes en functionele afbakening hebben een belangrijke rol gespeeld in deze voorbereidende fase. Het ERD en het use case diagram maakten de structuur van de applicatie en de interacties tussen gebruikers en systeem concreet. De MoSCoW-analyse hielp om duidelijke prioriteiten te stellen binnen de scope van het project, terwijl de prototypes richting gaven aan de verdere uitwerking.

Dit hoofdstuk vormt daarmee de brug tussen de probleemstelling en de effectieve realisatie van de applicatie. Op basis van deze analyse kon het project met een duidelijke functionele scope, een onderbouwde technische architectuur en een concrete planning verder worden uitgewerkt in de ontwikkelfase.

3. Realisatie

3.1. Inleiding

In dit hoofdstuk wordt de effectieve realisatie van het kennisdelingsplatform besproken. Waar in het vorige hoofdstuk de nadruk lag op de analyse, de technologiekeuzes en de functionele scope, wordt hier toegelicht hoe deze keuzes concreet zijn omgezet in een werkende toepassing. Daarbij wordt ingezoomd op de algemene opbouw van de applicatie, de belangrijkste gerealiseerde functionaliteiten en een aantal technische principes en patronen die tijdens de ontwikkeling werden toegepast. Het doel van dit hoofdstuk is niet om elk detail van de code te bespreken, maar wel om een duidelijk beeld te geven van de gerealiseerde oplossing en de manier waarop deze technisch werd opgebouwd.

3.2. Patronen

In dit onderdeel wordt toegelicht welke patronen tijdens de realisatie van het kennisdelingsplatform werden toegepast in zowel de frontend als de backend. Naast de ontwikkeling van de verschillende functionaliteiten werd ook veel aandacht besteed aan een duidelijke en gestructureerde opbouw van de toepassing. Daarom werd gebruikgemaakt van verschillende patronen en principes die bijdragen aan de onderhoudbaarheid, overzichtelijkheid en uitbreidbaarheid van de codebase.

Aangezien sommige van deze patronen een ingrijpende impact hadden op de structuur van de applicatie en zich over een groot deel van de codebase uitstrekten, werden deze in sterke mate in teamverband uitgewerkt. Om die reden ligt de focus in dit onderdeel vooral op de toegepaste patronen zelf en de meerwaarde ervan binnen het project.

3.2.1. Gebruikte patronen in de frontend

Onze frontend is opgebouwd volgens een feature-based structuur. Dat betekent dat de code niet in de eerste plaats gegroepeerd wordt per technisch type, maar vooral per functioneel domein.

Binnen features zitten bijvoorbeeld aparte delen voor topics, tags en feedback. Zo blijft alle logica die bij een bepaald onderdeel hoort dicht bij elkaar staan, wat de leesbaarheid en onderhoudbaarheid verbetert.

Binnen zo'n feature gebruiken we een *layered structure*. We onderscheiden vooral:

- pages: schermen of containercomponenten
- components: herbruikbare UI-onderdelen binnen een feature
- facade: laag tussen componenten en services
- services en data-access: communicatie met backend
- models: types en state-definities

Het belangrijkste patroon dat we in de frontend gebruiken, is het facade-patroon. Componenten spreken niet rechtstreeks met de API of met complexe businesslogica, maar via een facade. Die facade beheert de state, biedt computed waarden aan voor de UI, en bevat acties zoals laden, opslaan, filteren of publiceren. Daardoor blijven componenten relatief klein, ze focussen vooral op weergave, gebruikersinteractie en formulierbinding.

Voor statebeheer gebruiken we Angular signals. Elke facade heeft typisch een centrale state signal als single source of truth, aangevuld met computed waarden voor afgeleide UI-state. Dat zorgt ervoor dat we zonder extra externe state management library moeten werken.

Kort samengevat gebruikt onze frontend dus vooral een feature-based architectuur met een facade-patroon en signal-based state management. Dat geeft ons een structuur die schaalbaar, onderhoudbaar en duidelijk leesbaar is naarmate de applicatie groeit.

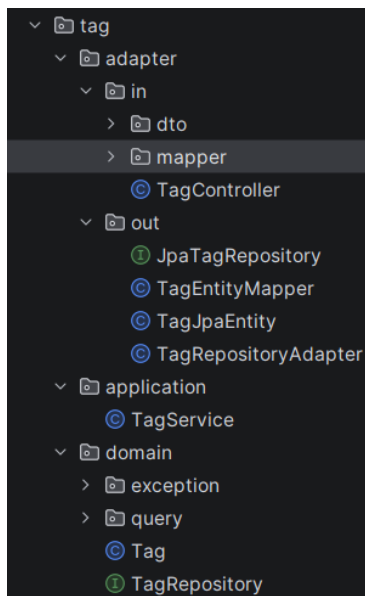
3.2.2. Gebruikte patronen in de backend

In de backend werd gekozen voor een combinatie van Domain-Driven Design (DDD) en een hexagonale architectuur. Domain-Driven Design vertrekt vanuit het idee dat de structuur van de software nauw moet aansluiten bij het functionele domein van de toepassing. Hierdoor wordt de businesslogica niet verspreid over de applicatie, maar ondergebracht in duidelijke domeinmodellen en domeinservices.

Aanvullend daarop werd een hexagonale architectuur toegepast. Binnen deze architectuur staat de kernlogica centraal, terwijl externe onderdelen zoals databanktoegang, API's of andere technische koppelingen via aparte adapters verbonden worden. Dit zorgt ervoor dat de kern van de applicatie minder afhankelijk wordt van specifieke technologieën.

De combinatie van beide patronen maakt de backend beter onderhoudbaar, testbaar en uitbreidbaar. Door een duidelijke scheiding tussen domeinlogica en infrastructuur kan de applicatie op een meer gestructureerde manier verder evolueren.

In onze concrete implementatie is dit terug te zien in de opsplitsing tussen adapter, application en domain (zie Figuur 3). De domain-laag bevat de kern van de applicatie, zoals domeinmodellen, businessregels en repositorycontracten. De application-laag bevat de services die de verschillende use cases aansturen en de interactie tussen domeinobjecten coördineren. De adapter-laag vormt de verbinding met de buitenwereld en bevat onder andere REST-controllers, DTO's, mappers en repository-implementaties voor de databanktoegang.



Figuur 3: Indeling van de aggregaat Tag in de backend-mappenstructuur.

3.3. Functionaliteiten

In dit hoofdstuk verschuift de focus van de gezamenlijke projectvisie naar mijn individuele bijdrage aan het kennisdelingsplatform. Hoewel de applicatie het resultaat is van een nauwe samenwerking, zijn de hierna beschreven functionaliteiten voor het grootste deel door mijzelf ontwikkeld en geïmplementeerd.

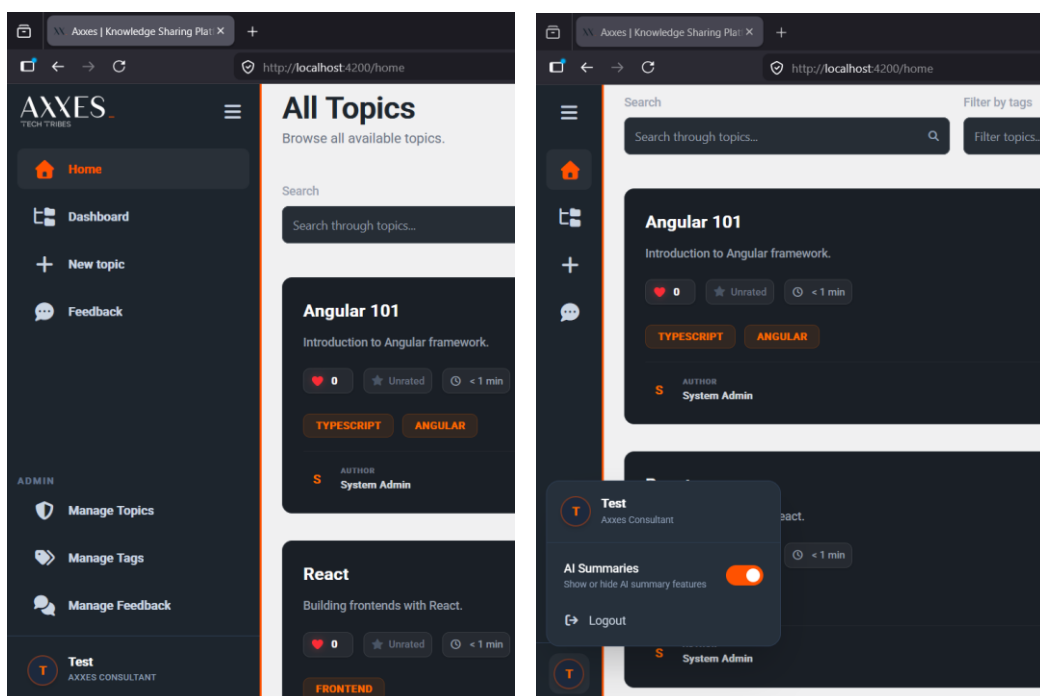
Per functionaliteit licht ik kort de toegevoegde waarde toe en leg ik uit hoe deze binnen het grotere geheel past. Daarnaast bied ik inzicht in de technische realisatie, waarbij ik de gemaakte keuzes en de gebruikte technologieën onderbouw. Omwille van interne richtlijnen van Axxes wordt in dit document geen letterlijke broncode opgenomen. Technische onderdelen worden waar relevant daarom toegelicht met vereenvoudigde pseudocode.

Dit hoofdstuk vormt zo een helder overzicht van mijn persoonlijke aandeel in het project en de technische stappen die ik tijdens het proces heb gezet.

3.3.1. Navigatiesidebar

Binnen de frontend van het kennisdelingsplatform werkte ik een navigatiesidebar uit om een duidelijke en consistente navigatiestructuur te voorzien over de verschillende pagina's en functionaliteiten. Ik realiseerde de sidebar als een herbruikbare Angular-component en integreerde deze in de algemene applicatielayout.

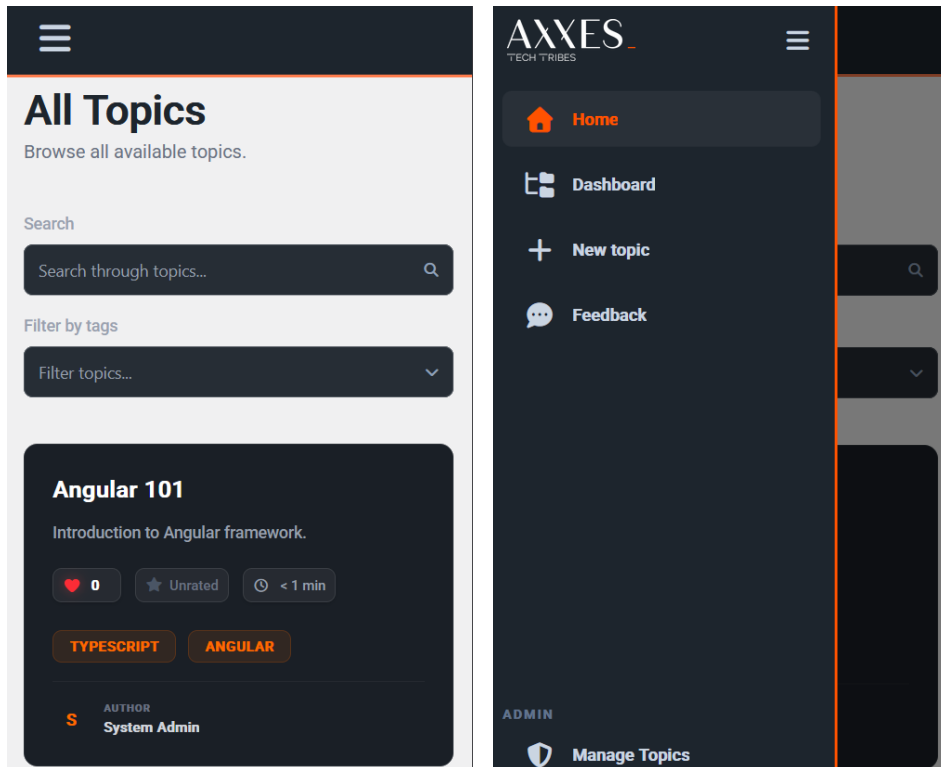
Via de sidebar (zie Figuur 4 en Figuur 5) kan de gebruiker navigeren naar de belangrijkste pagina's, instellingen raadplegen en uitloggen. Bij de uitwerking hield ik rekening met responsiviteit.



Figuur 4: Standaardweergave van de navigatiesidebar

Figuur 5: Ingeklapte navigatiesidebar met toegang tot gebruikersinstellingen

Dezelfde navigatiestructuur werd ook aangepast voor mobiele schermformaten. Op kleinere schermen wordt de sidebar compacter weergegeven, zodat de navigatie bruikbaar blijft zonder de inhoud van de pagina te veel te beperken (zie Figuur 6 en Figuur 7).



Figuur 6: Mobiele weergave van de navigatiesidebar

Figuur 7: Mobiele weergave van de uitgeklapte navigatie

```

SERVICE NavigatieSidebar

VARIABELE sidebarIngeklapt = false
VARIABELE mobielScherf = false

CONSTANTE volledigeBreedte = 300
CONSTANTE minimaleBreedte = 80

FUNCTIE berekenInhoudsMarge()
  ALS mobielScherf = true
    GEEF 0 TERUG
  ANDERS ALS sidebarIngeklapt = true
    GEEF minimaleBreedte TERUG
  ANDERS
    GEEF volledigeBreedte TERUG
  EINDE ALS
EINDE FUNCTIE

BIJ OPSTARTEN
  CONTROLEER of scherm breedte kleiner is dan 768px

  ALS scherm mobiel is
    mobielScherf = true
    sidebarIngeklapt = true
  ANDERS
    mobielScherf = false
    sidebarIngeklapt = false
  EINDE ALS
EINDE

FUNCTIE wisselSidebarStatus()
  sidebarIngeklapt = NIET sidebarIngeklapt
EINDE FUNCTIE

EINDE SERVICE

```

Tabel 7: Pseudocode van de navigatiesidebar

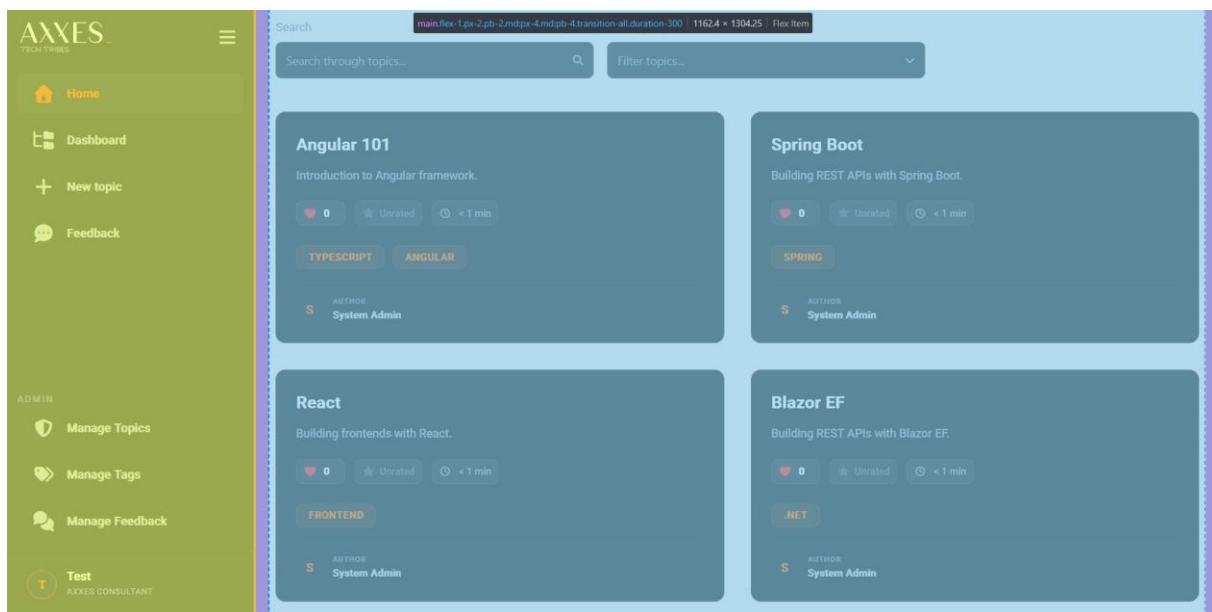
Onderstaand pseudocodefragment (zie Tabel 7) toont de logica achter de navigatiesidebar. Hierin wordt bijgehouden of de sidebar ingeklapt is en of de applicatie op een mobiel scherm wordt gebruikt. Op basis daarvan wordt bepaald hoeveel ruimte beschikbaar is voor de inhoud van de pagina. Daarnaast wordt ook beschreven hoe de sidebar standaard anders reageert op desktop en mobiel, en hoe de gebruiker deze kan open en dichtklappen.

3.3.2. Algemene layout van de applicatie

Naast de navigatiesidebar werkte ik ook aan de algemene layout van de applicatie. Deze zorgt ervoor dat de inhoud van elke pagina op een consistente manier naast de navigatie wordt weergegeven en niet achter de sidebar terechtkomt.

Mijn bijdrage lag vooral in het voorzien van het juiste vak voor de pagina-inhoud en in het bepalen van de algemene afmetingen, marges en ruimteverdeling. Daardoor was er voor elke pagina voldoende ruimte beschikbaar om de content correct weer te geven. De verdere invulling van die ruimte, zoals bijvoorbeeld formulieren die meer gecentreerd stonden of extra witruimte links en rechts kregen, werd nadien binnen de styling van die specifieke pagina's verder uitgewerkt.

Daarnaast werd de layout ook afgestemd op verschillende schermgroottes. Wanneer de navigatiesidebar ingeklapt wordt of wanneer de applicatie op mobiel gebruikt wordt, past de beschikbare ruimte voor de inhoud zich automatisch aan.



Figuur 8: Algemene applicatielayout met navigatiesidebar en hoofdinhoud

Dit figuur (zie Figuur 8) toont de algemene lay-out van de applicatie. Links is de navigatiesidebar zichtbaar, terwijl rechts de hoofdinhoud van de pagina wordt weergegeven. In deze opbouw is te zien hoe de contentzone afgestemd is op de ruimte die door de navigatie wordt ingenomen.

3.3.3. Tailwind CSS-integratie

Binnen de frontend van het kennisdelingsplatform heb ik meegewerkt aan de integratie van Tailwind CSS. Deze keuze maakte het mogelijk om de styling van de applicatie consistent en efficiënter op te bouwen via utility classes.

Er werd voor Tailwind CSS gekozen omdat deze aanpak zeer direct, flexibel en modulair werkt. In vergelijking met Bootstrap laat Tailwind meer vrijheid toe om een eigen stijlsysteem op te bouwen en sneller consistente componenten uit te werken. Ook tegenover traditionele CSS-bestanden biedt deze aanpak een voordeel, omdat styling sneller toegepast en aangepast kan worden zonder grote aparte stylesheets te moeten onderhouden. Binnen het project werd daarom ook gewerkt met een eigen Tailwind-thema en herbruikbare klassen.

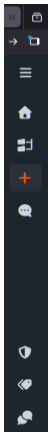
Mijn bijdrage bestond uit het integreren van Tailwind CSS in het hoofdproject en het omzetten van bestaande CSS-bestanden naar Tailwind.

3.3.4. Font Awesome-integratie

Binnen de frontend van het kennisdelingsplatform heb ik de Font Awesome iconenbibliotheek geïntegreerd. Hierdoor konden iconen op een consistente manier gebruikt worden binnen verschillende onderdelen van de applicatie, zoals de navigatie (zie Figuur 9) en andere interactieve componenten.

Er werd voor Font Awesome gekozen omdat deze bibliotheek eenvoudig te gebruiken is en een brede keuze aan iconen aanbiedt die visueel sterker zijn dan andere iconenbibliotheken.

Mijn bijdrage bestond uit het toevoegen en configureren van deze bibliotheek binnen het Angular-project. Deze integratie verbeterde de visuele duidelijkheid van de interface en maakte acties en navigatieopties sneller herkenbaar voor de gebruiker.



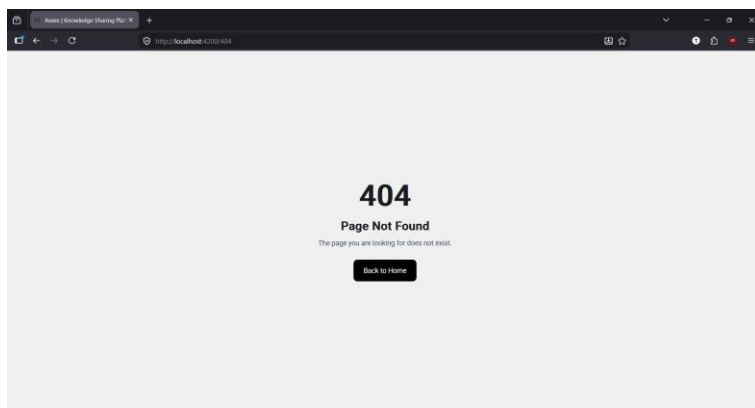
Figuur 9: Gebruik van Font Awesome-iconen in de interface

3.3.5. Error handling en redirectpagina's in de frontend

Binnen de frontend van het kennisdelingsplatform werkte ik aan de foutafhandeling, zodat gebruikers bij foutieve of niet-toegestane acties niet op een gebroken pagina of een technische crash terechtkomen. In plaats daarvan worden zij doorgestuurd naar een aangepaste foutpagina die duidelijk aangeeft wat er fout liep.

Concreet werden in de applicatie verschillende foutpagina's voorzien, zoals voor ongeautoriseerde toegang, verboden toegang en niet-bestaande pagina's (zie Figuur 10). Hierdoor krijgt de gebruiker steeds gepaste feedback wanneer een route niet toegankelijk is of wanneer een ongeldige URL wordt opgevraagd.

Mijn bijdrage lag zowel in het uitwerken van deze foutpagina's in de frontend als in de bijhorende routeringslogica.



Figuur 10: Voorbeeld van een aangepaste foutpagina

DEFINIEER routes voor:

- startpagina
- 401 ongeautoriseerd
- 403 verboden
- 404 niet gevonden

ALS gebruiker niet aangemeld is

STUUR gebruiker door naar foutpagina 401

ANDERS

GEEF toegang tot beveiligde routes

EINDE ALS

VOOR bepaalde routes

CONTROLEER extra rechten, zoals moderatorrechten

ALS gebruiker deze rechten niet heeft

STUUR gebruiker door naar foutpagina 403

EINDE ALS

ALS een route niet bestaat

STUUR gebruiker door naar foutpagina 404

EINDE ALS

Tabel 8: Pseudocode van de redirect logica

Onderstaand pseudocodefragment (zie Tabel 8) toont vereenvoudigd hoe de foutafhandeling en routeringslogica in de frontend werden opgebouwd. Afhankelijk van de route, de sessiestatus en de rechten van de gebruiker wordt bepaald of toegang wordt verleend of dat een aangepaste foutpagina wordt weergegeven.

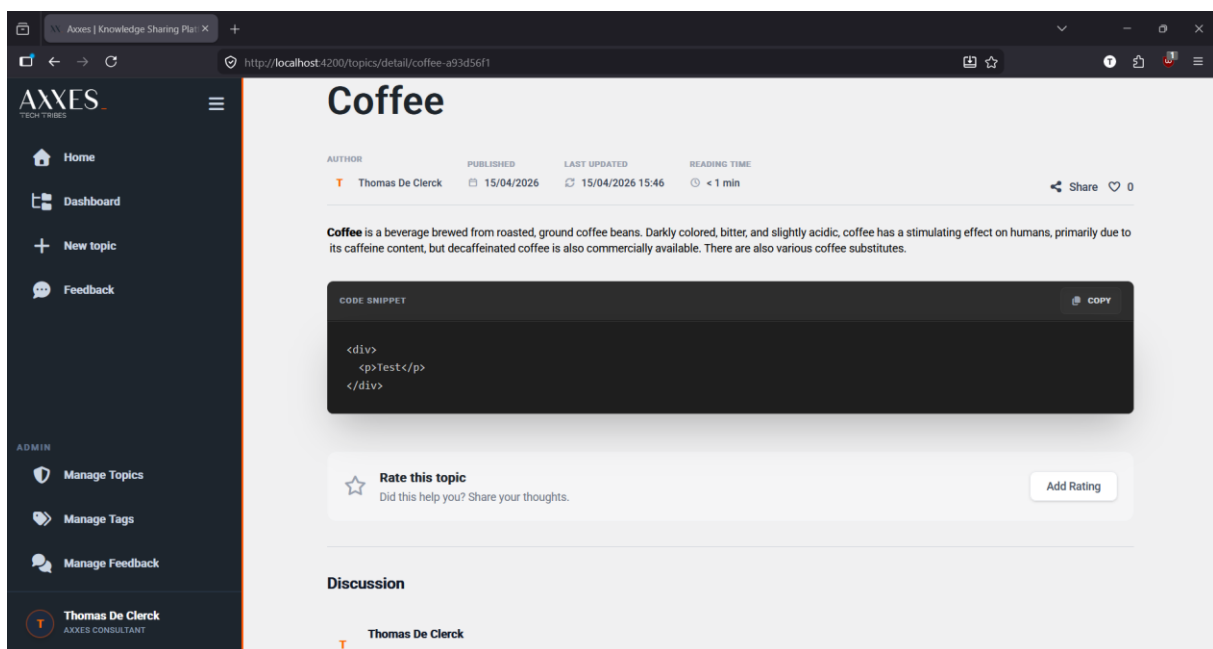
3.3.6. Topicbeheer

Binnen het kennisdelingsplatform werkte ik aan verschillende onderdelen van het topicbeheer. Aangezien topics de kern van de applicatie vormen, was het belangrijk dat deze niet alleen correct weergegeven konden worden, maar ook beheerd konden worden via acties zoals verwijderen, publiceren en archiveren. In de volgende onderdelen worden deze realisaties verder toegelicht.

3.3.6.1. Detailpagina van topics

Binnen het topicbeheer ontwikkelde ik de detailpagina van topics (zie Figuur 11). Deze pagina laat gebruikers toe om de volledige inhoud van een topic op een gestructureerde manier te raadplegen. Omdat topics de centrale inhoudseenheden van het platform vormen, was het belangrijk dat alle relevante informatie hier op een duidelijke manier werd samengebracht.

Mijn bijdrage lag in het uitwerken van deze detailpagina binnen de frontend, met aandacht voor de opmaak, positionering van de inhoud en de integratie van gekoppelde elementen.



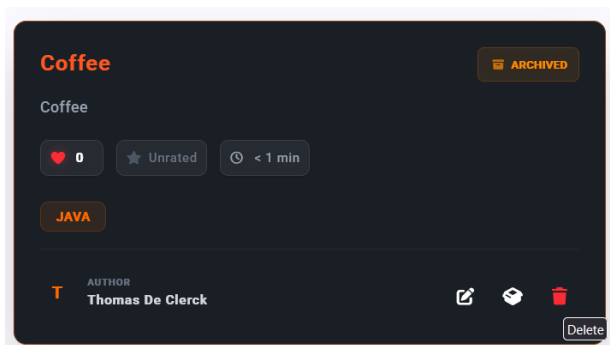
Figuur 11: Detailpagina van een topic

3.3.6.2. Delete-functionaliiteit

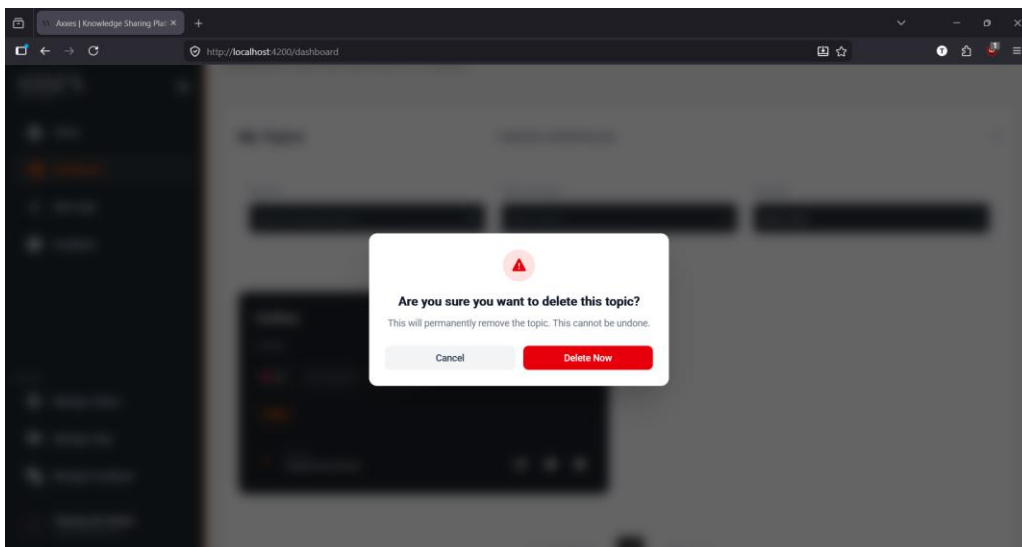
Naast de detailweergave werkte ik ook aan de delete-functionaliiteit voor topics. Deze actie werd voorzien via de moderatorpagina, zodat moderators topics kunnen verwijderen wanneer deze foutief, ongewenst of niet langer relevant zijn. Deze werking geldt ook voor topics die door gebruikers zelf werden aangemaakt en later beheerd of verwijderd moeten kunnen worden.

Deze functionaliteit vormde het startpunt van mijn werk aan de backendrefactor en was de eerste use case die binnen de nieuwe structuur werd uitgewerkt. Daardoor gebruikte ik delete als concrete testcase om de vernieuwde backend aanpak te onderzoeken, op te bouwen en te valideren.

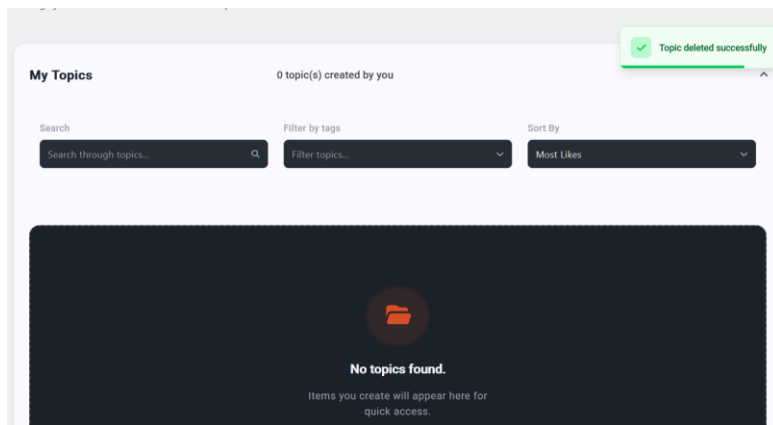
Mijn bijdrage lag zowel in het voorzien van de delete-actie binnen de moderatoromgeving als in het uitwerken en testen van de bijhorende backend logica.



Figuur 12: Topic in het dashboard met delete-actie



Figuur 13: Bevestigingsmodal voor het verwijderen van een topic



Figuur 14: Bevestiging na succesvolle verwijdering van een topic

De delete-functionaliteit kan worden gestart vanuit het dashboard van de gebruiker, waar de verwijderactie bij het topic beschikbaar is (zie Figuur 12). Vooraleer het topic definitief verwijderd wordt, verschijnt een bevestigingsmodal waarin de gebruiker de actie nog kan annuleren of bevestigen (zie Figuur 13). Na een succesvolle verwijdering wordt een bevestiging getoond, zodat duidelijk is dat de actie correct werd uitgevoerd (zie Figuur 14).

```

ALS gebruiker op "verwijderen" klikt
  TOON bevestigingsvenster
EINDE ALS

ALS gebruiker de verwijdering bevestigt
  CONTROLEER of gebruiker de juiste rechten heeft
  CONTROLEER of het topic bestaat

  ALS beide voorwaarden geldig zijn
    VERWIJDER het topic
    TOON bevestiging aan de gebruiker
    VERVERS het overzicht van topics
  ANDERS
    TOON foutmelding of foutpagina
  EINDE ALS
EINDE ALS

```

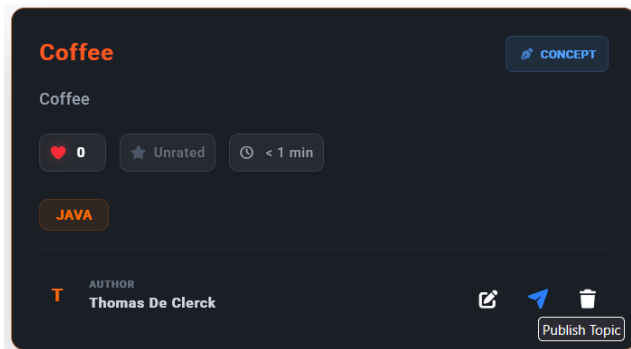
Tabel 9: Pseudocodefragment van de delete-functionaliteit

Onderstaand pseudocodefragment toont (zie Tabel 9) vereenvoudigd hoe de delete-functionaliteit werd opgebouwd. Eerst krijgt de gebruiker een bevestigingsvenster te zien. Wanneer de actie bevestigd wordt, controleert het systeem of de gebruiker de juiste rechten heeft en of het topic bestaat. Als aan deze voorwaarden voldaan is, wordt het topic verwijderd en krijgt de gebruiker een bevestiging te zien.

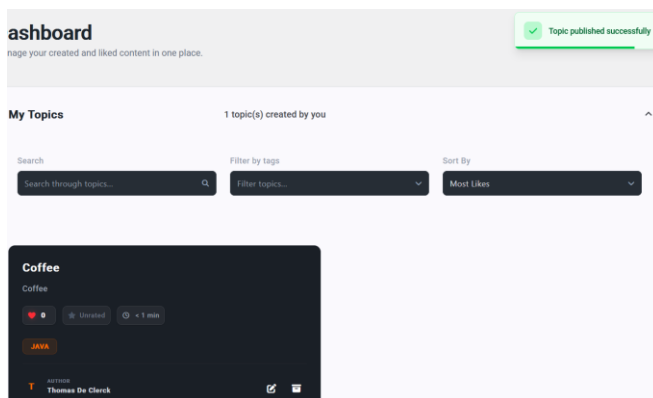
3.3.6.3. Publicatiefunctie

Binnen het topicbeheer implementeerde ik ook de publicatiefunctie. Deze functionaliteit maakt het mogelijk om topics vanuit de moderatorpagina te publiceren, zodat ze zichtbaar en beschikbaar worden binnen het platform. Daarnaast geldt deze werking ook voor topics die door de gebruiker zelf werden aangemaakt en nadien verder beheerd of afgewerkt worden.

De publicatiefunctie is belangrijk omdat niet alle topics onmiddellijk publiek beschikbaar mogen zijn. Via deze werking kan een topic eerst nagekeken en beheerd worden voordat het effectief gepubliceerd wordt.



Figuur 15: Publicatieactie voor een topic in het dashboard



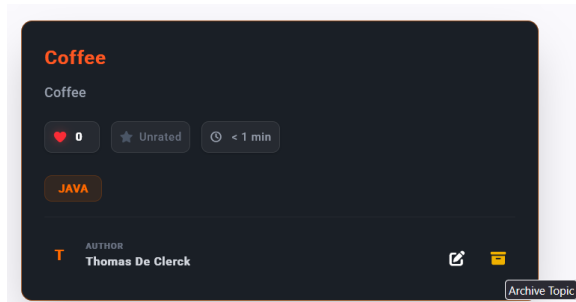
Figuur 16: Bevestiging na succesvolle publicatie van een topic

Vanuit het dashboard kan een topic vanuit conceptstatus gepubliceerd worden (zie Figuur 15). Na het publiceren krijgt de gebruiker een bevestiging dat het topic succesvol publiek beschikbaar werd gemaakt (zie Figuur 16).

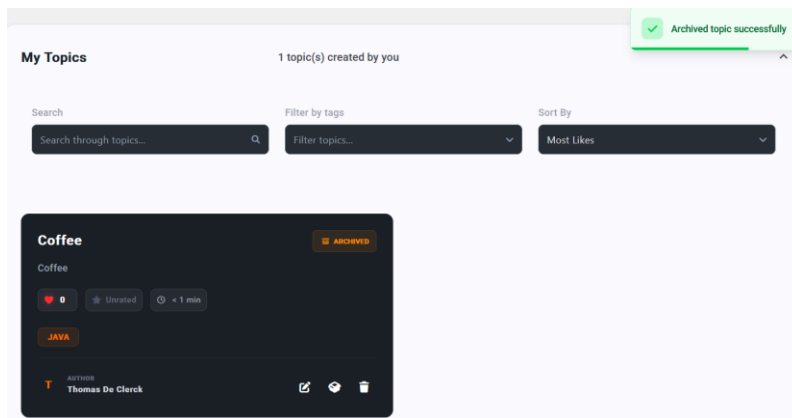
3.3.6.4. Archiveringslogica

Een laatste onderdeel binnen het topicbeheer was de archiveringslogica. Deze functionaliteit laat toe om topics uit de actieve stroom te halen zonder ze definitief te verwijderen.

De basis van deze logica was al aanwezig, maar na de implementatie van de publicatiefunctie moest deze werking verder aangepast worden. Mijn bijdrage lag in het uitbreiden van deze bestaande logica met extra validatie en autorisatie, zodat concepttopics niet rechtstreeks gearchiveerd kunnen worden en onbevoegde gebruikers deze actie niet via een rechtstreekse link kunnen uitvoeren.



Figuur 17: Archiveringsactie voor een topic in het dashboard



Figuur 18: Bevestiging na succesvolle archivering van een topic

Vanuit het dashboard kan een gebruiker een eigen topic archiveren, zodat het topic uit de actieve stroom wordt gehaald zonder het definitief te verwijderen (zie Figuur 17). Na een succesvolle archivering wordt een bevestiging weergegeven, zodat duidelijk is dat de actie correct werd uitgevoerd (zie Figuur 18).

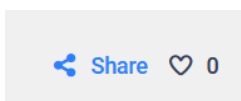
3.3.7. Interactie en sociale functionaliteiten

Binnen het kennisdelingsplatform werkte ik ook aan verschillende interactieve functionaliteiten die de betrokkenheid van gebruikers verhogen. Deze onderdelen maken het mogelijk om topics te delen, te liken en te beoordelen. Daarnaast werd ook ondersteuning voorzien om gelikete topics vanuit de backend op te halen. In de volgende onderdelen worden deze realisaties kort toegelicht.

3.3.7.1. Share-functionaliteit

Binnen dit onderdeel ontwikkelde ik de share-functionaliteit (zie Figuur 19), waarmee gebruikers topics eenvoudig kunnen delen met anderen. Deze functionaliteit maakt het makkelijker om relevante kennis verder te verspreiden binnen of buiten het platform.

Mijn bijdrage lag in het uitwerken van deze functionaliteit binnen de frontend, zodat gebruikers op een eenvoudige manier een topic kunnen delen vanuit de applicatie.

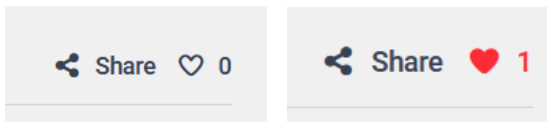


Figuur 19: Share-actie op de detailpagina van een topic

3.3.7.2. Like-functionaliteit

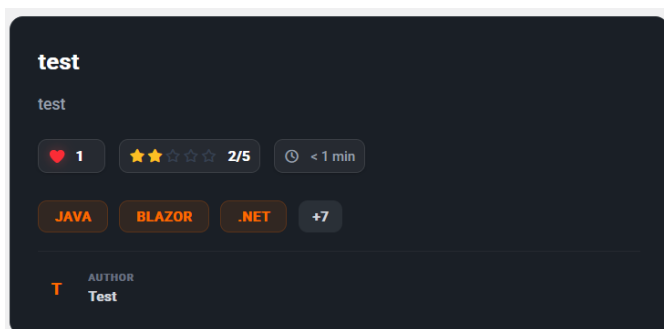
Een tweede onderdeel binnen deze categorie was de like-functionaliteit op topics. Hiermee kunnen gebruikers waardering tonen voor inhoud die zij nuttig of relevant vinden. Dit draagt bij aan de interactie binnen het platform en maakt het mogelijk om populaire topics sneller te herkennen.

De basis van deze functionaliteit was al uitgewerkt, maar in het kader van de backendrefactor heb ik deze opnieuw geïntegreerd binnen de vernieuwde structuur.



Figuur 20: Like-actie op de detailpagina van een topic

Figuur 21: Visuele feedback na het liken van een topic



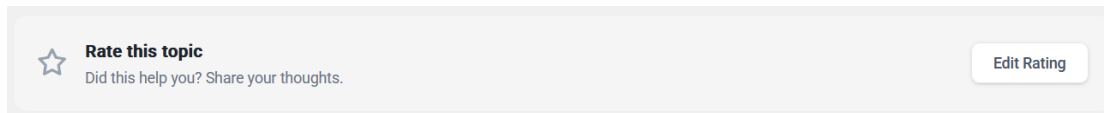
Figuur 22: Weergave van het aantal likes in het dashboard

Figuur 20 en Figuur 21 tonen hoe een gebruiker een topic kan liken op de detailpagina. Het aantal likes wordt daarnaast ook weergegeven in het dashboard, zodat populaire topics sneller herkenbaar zijn (zie Figuur 22).

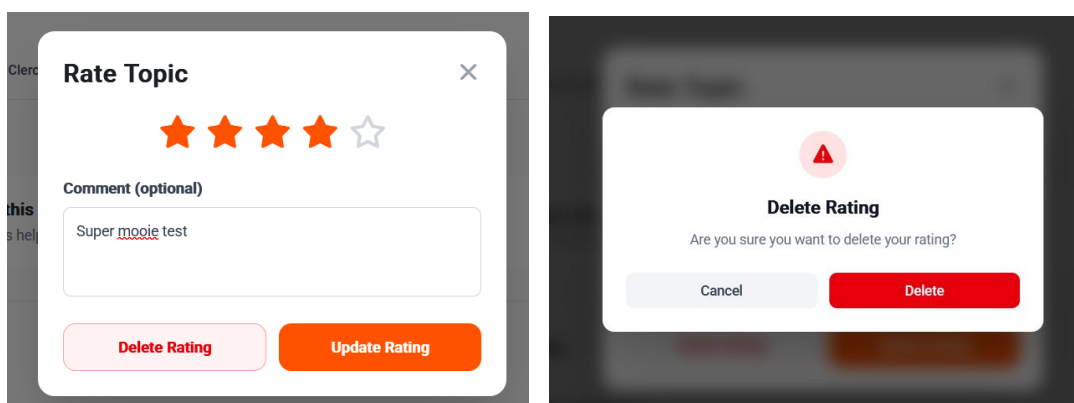
3.3.7.3. Rating-functionaliiteit

Naast likes werkte ik ook aan de rating-functionaliiteit, waarmee gebruikers een waardering aan topics kunnen geven. Deze functionaliteit maakt het mogelijk om inhoud niet alleen te liken, maar ook gericht te beoordelen.

Ook hier was de basis al aanwezig, maar werd de functionaliteit door mij opnieuw geïntegreerd binnen de backendrefactor. Daarnaast werkte ik ook aan het verwijderen van ratings, waarbij de bevestigingsmodal verder werd aangepast om deze actie correct te ondersteunen.



Figuur 23: Toegang tot ratingbeheer op de detailpagina van een topic



Figuur 24: Delete-actie voor een rating

Figuur 25: Bevestiging voor het verwijderen van een rating

Figuur 23 en Figuur 24 tonen hoe een gebruiker via de detailpagina een rating kan beheren en verwijderen. Voor het definitief verwijderen wordt een bevestigingsmodal weergegeven (zie Figuur 25), zodat de actie bewust bevestigd moet worden.

3.3.8. Feedbackpagina met EmailJS

Binnen de applicatie ontwikkelde ik een eerste versie van de feedbackpagina, waarmee gebruikers eenvoudig feedback konden doorsturen. In deze oorspronkelijke uitwerking werd gebruikgemaakt van EmailJS om ingezonden feedback rechtstreeks te verzenden.

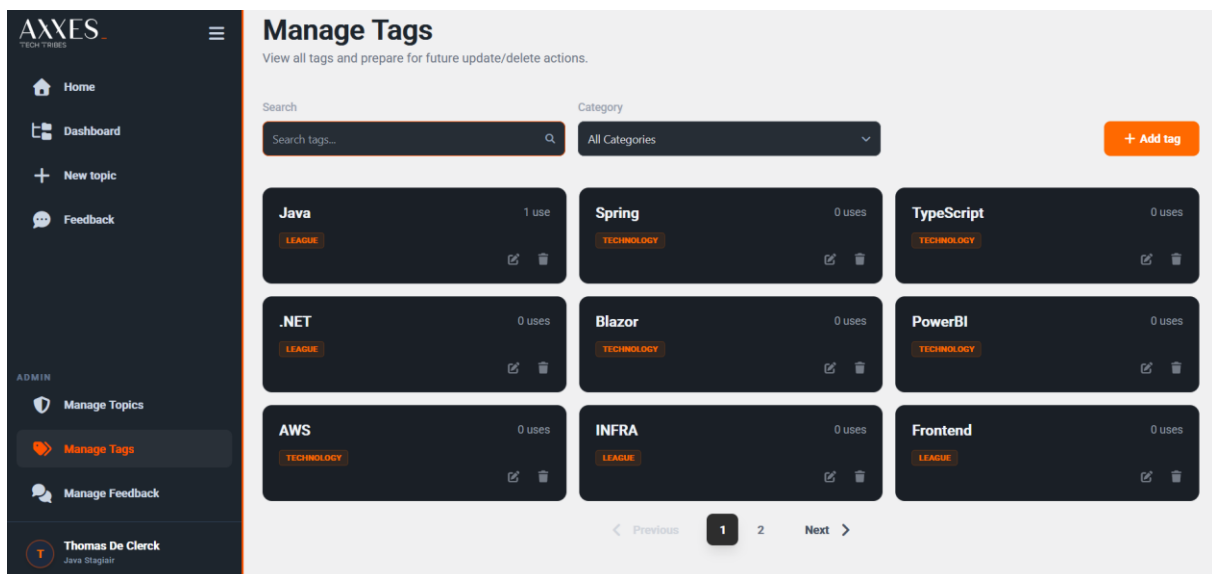
Mijn bijdrage lag in het opzetten van deze feedbackflow en in het uitwerken van de bijhorende frontendpagina. Daarmee werd een eerste werkende basis voorzien voor het verzamelen en verwerken van feedback binnen het platform.

In een latere fase van het project werd deze functionaliteit verder herwerkt, waarbij werd overgestapt naar een aanpak waarbij feedback in de databank wordt opgeslagen. Hoewel de uiteindelijke implementatie daardoor verschilt van mijn oorspronkelijke versie, bouwde deze verdere uitwerking wel voort op de basis die reeds aanwezig was.

3.3.9. Tagbeheer CRUD

Een ander onderdeel waaraan ik werkte, was het beheer van tags binnen het platform. Tags spelen een belangrijke rol in het structureren en terugvinden van topics, omdat ze gebruikers helpen om inhoud beter te classificeren en sneller relevante onderwerpen terug te vinden. Daarom werd functionaliteit voorzien om tags aan te maken, te raadplegen, te wijzigen, te verwijderen en het gebruik ervan binnen de applicatie op te volgen.

Dit onderdeel werd volledig door mij gerealiseerd. Daarbij werkte ik zowel de frontend als de backend uit, inclusief de nodige validaties om foutieve invoer correct op te vangen. Daarnaast voorzag ik ook functionaliteit om het gebruik van tags binnen de applicatie op te volgen.



Figuur 26: Beheerscherm voor tags

Figuur 26 toont het beheerscherm voor tags. Via deze pagina kunnen tags gefilterd, aangemaakt, bewerkt en verwijderd worden. Voor het aanmaken, wijzigen en verwijderen wordt gebruikgemaakt van modalvensters.

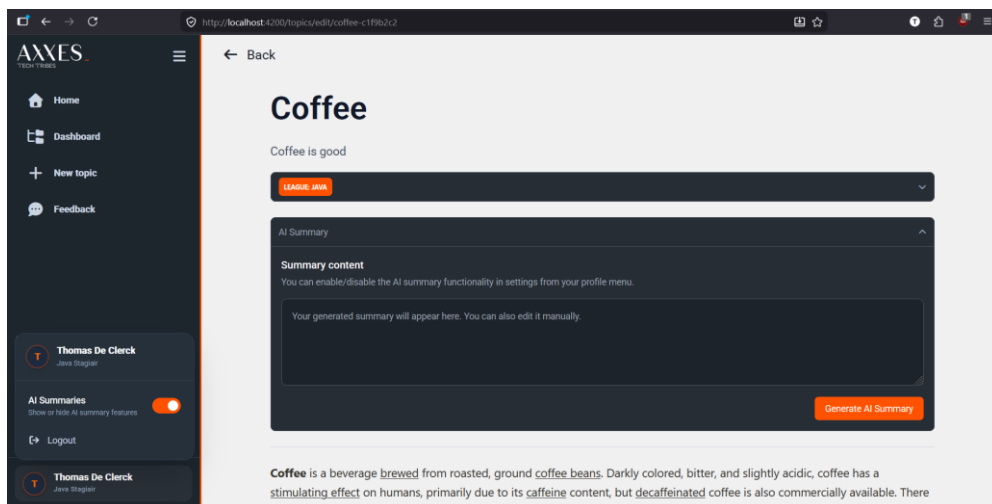
3.3.10. AI-samenvatter met AWS Lambda (TL;DR-functionaliteit)

Een van de meer geavanceerde functionaliteiten die ik realiseerde, is de AI-samenvatter binnen het kennisdelingsplatform. Deze functionaliteit maakt het mogelijk om automatisch een korte samenvatting van een topic te genereren, zodat gebruikers sneller de essentie van een onderwerp kunnen raadplegen.

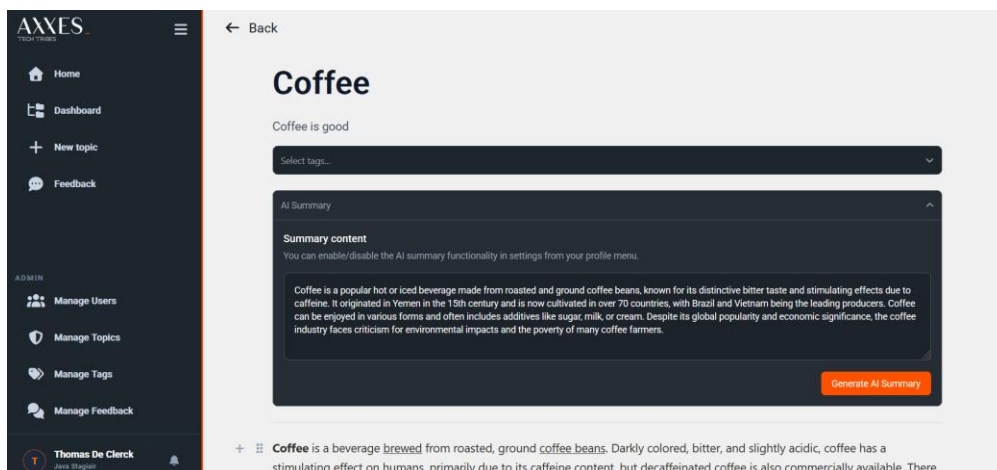
Voor de technische uitwerking werd gebruikgemaakt van AWS Lambda en het Amazon Nova Pro-model. Mijn bijdrage lag niet alleen in de integratie van deze AI-functionaliteit, maar ook in het uitbreiden van de backend met een aparte service, de nodige databankvelden, validatie en uitgebreide foutafhandeling.

Voorafgaand aan de implementatie doorliep ik ook een volledige onderzoeksfase, waarin ik de haalbaarheid en de meest geschikte technische aanpak onderzocht. Deze bevindingen en het voorgestelde concept werden daarna ook toegelicht tijdens een meeting, zodat de verdere realisatie afgestemd kon worden binnen het project.

Daardoor werd niet alleen de AI-integratie zelf gerealiseerd, maar ook de nodige technische omkadering om deze op een betrouwbare en onderhoudbare manier in het platform op te nemen.



Figuur 27: AI-samenvatter op de detailpagina van een topic



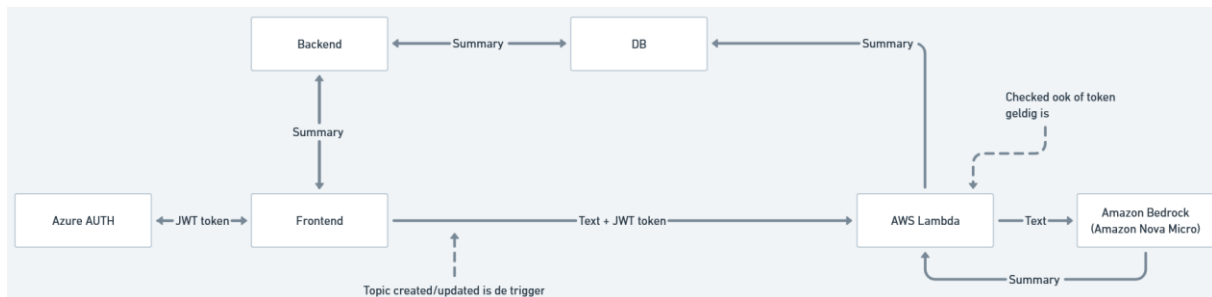
Figuur 28: Gegeneerde AI-samenvatting in de frontend

De volgende figuren tonen hoe de AI-samenvatter geïntegreerd werd in de frontend. Via deze functionaliteit kan een gebruiker een automatisch gegenereerde samenvatting van een topic raadplegen (zie Figuur 27 en Figuur 28).

Communicatiestroom van de AI-samenvatter

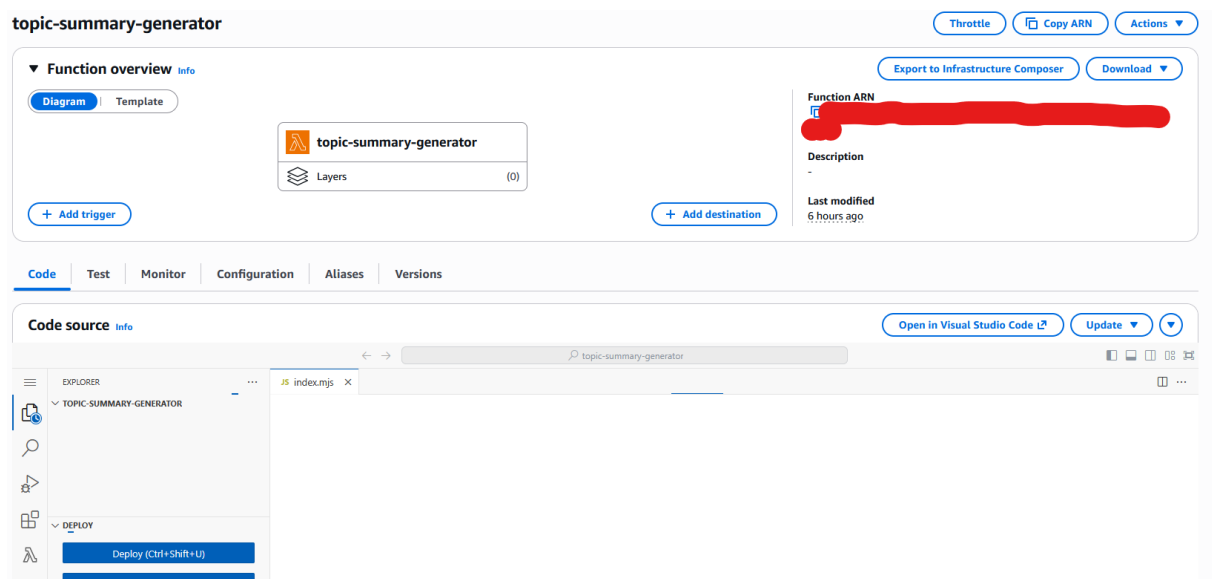
Onderstaande figuur toont de communicatiestroom van de AI-samenvatter binnen het kennisdelingsplatform. Hierin is zichtbaar hoe de frontend, backend, databank en AWS-componenten samenwerken om automatisch een samenvatting van een topic te genereren.

Wanneer een topic wordt aangemaakt of aangepast, wordt de relevante inhoud samen met een JWT-token doorgestuurd. Via AWS Lambda wordt het token gecontroleerd en wordt de tekst doorgegeven aan Amazon Bedrock, waar het AI-model de samenvatting genereert. Deze samenvatting wordt vervolgens teruggestuurd, opgeslagen in de databank en opnieuw beschikbaar gemaakt voor de frontend via de backend.



Figuur 29: Communicatiestroom van de AI-samenvatter

Dit diagram (zie Figuur 29) toont hoe de frontend, backend, databank en AWS samenwerken binnen de AI-samenvatter.

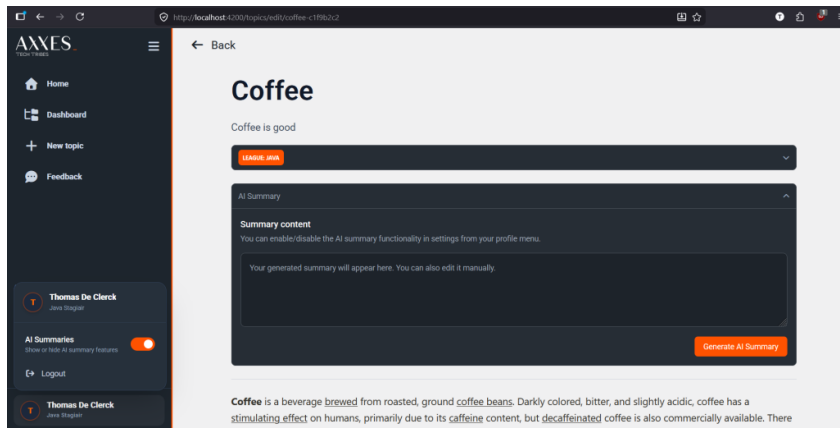


Figuur 30: AWS Lambda-functie van de AI-samenvatter

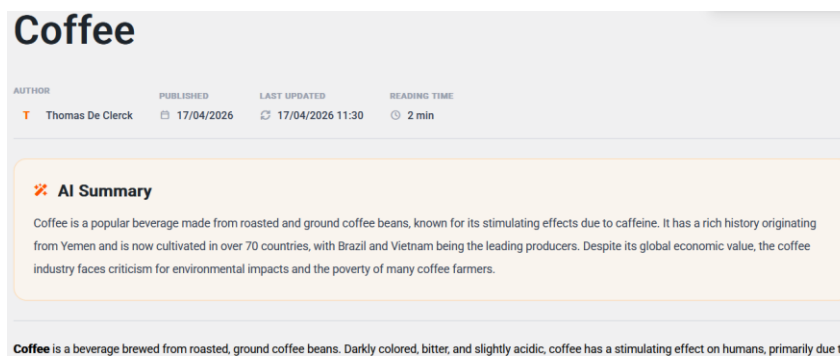
Figuur 30 toont de AWS Lambda-functie die instaat voor de verwerking van de aanvraag binnen de AI-samenvatter.

3.3.11. Instelling voor de AI-samenvatter

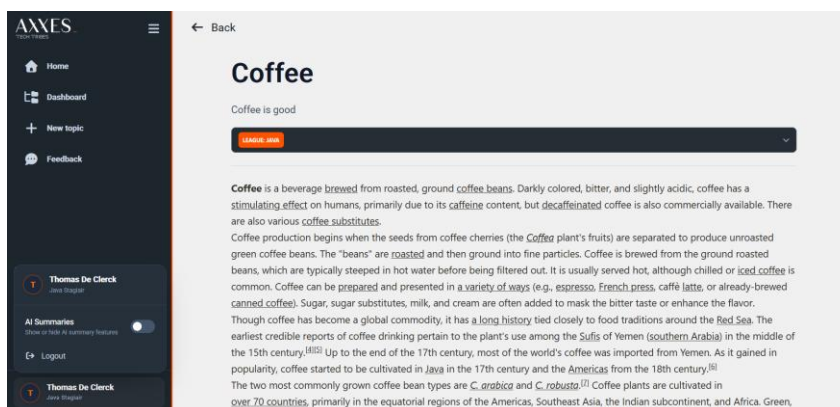
Naast de AI-samenvatter zelf werkte ik ook aan één instelling waarmee deze functionaliteit volledig actief of inactief kan worden gezet binnen de applicatie. Wanneer deze actief is, wordt de samenvatter weergegeven, wanneer ze inactief is, is deze niet zichtbaar.



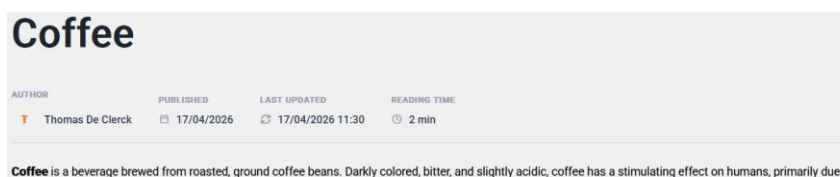
Figuur 31: Ingeschakelde instelling voor de AI-samenvatter



Figuur 32: AI-samenvatter zichtbaar op de detailpagina



Figuur 33: Uitgeschakelde instelling voor de AI-samenvatter

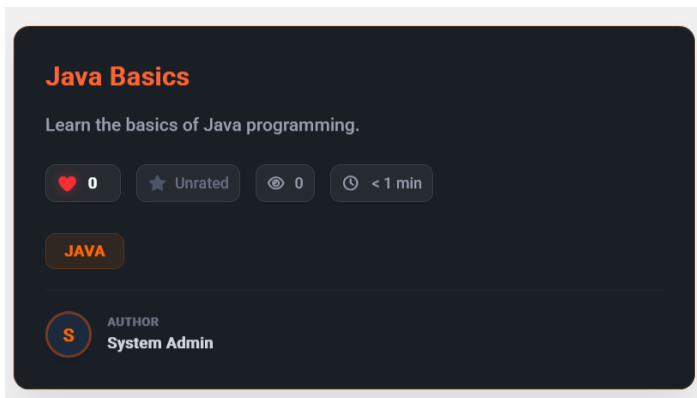


Figuur 34: AI-samenvatter verborgen op de detailpagina

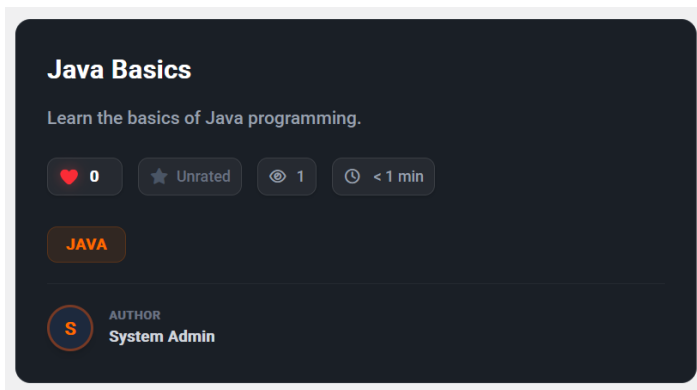
Figuur 31 en Figuur 32 tonen de situatie waarin de instelling voor de AI-samenvatter is ingeschakeld. In dat geval wordt de AI-samenvatter zichtbaar weergegeven op de detailpagina van een topic. Figuur 33 en Figuur 34 tonen de situatie waarin de instelling is uitgeschakeld. Daarbij wordt de AI-samenvatter volledig verborgen op de detailpagina.

3.3.12. Topic weergaven

Een bijkomende functionaliteit binnen het platform is de view counter van topics. Deze houdt bij hoe vaak een topic bekeken werd, zodat er meer inzicht ontstaat in het gebruik van het platform. Mijn bijdrage lag in het registreren en weergeven van deze teller binnen de applicatie.



Figuur 35: Aantal weergaven van een topic vóór het openen

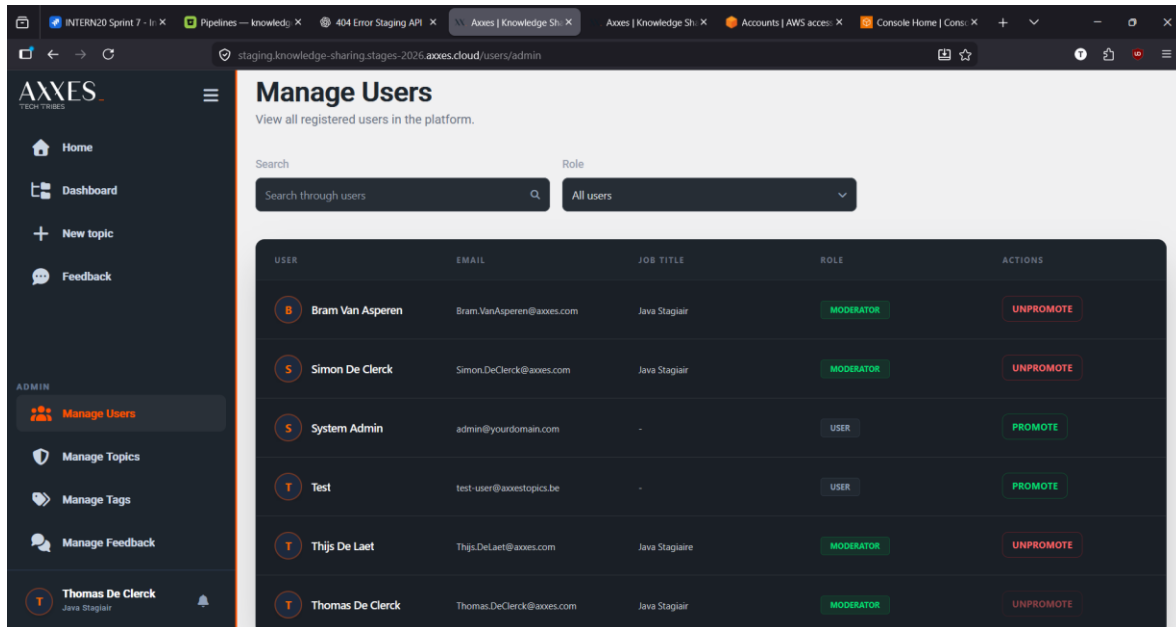


Figuur 36: Bijgewerkte view counter na het bekijken van een topic

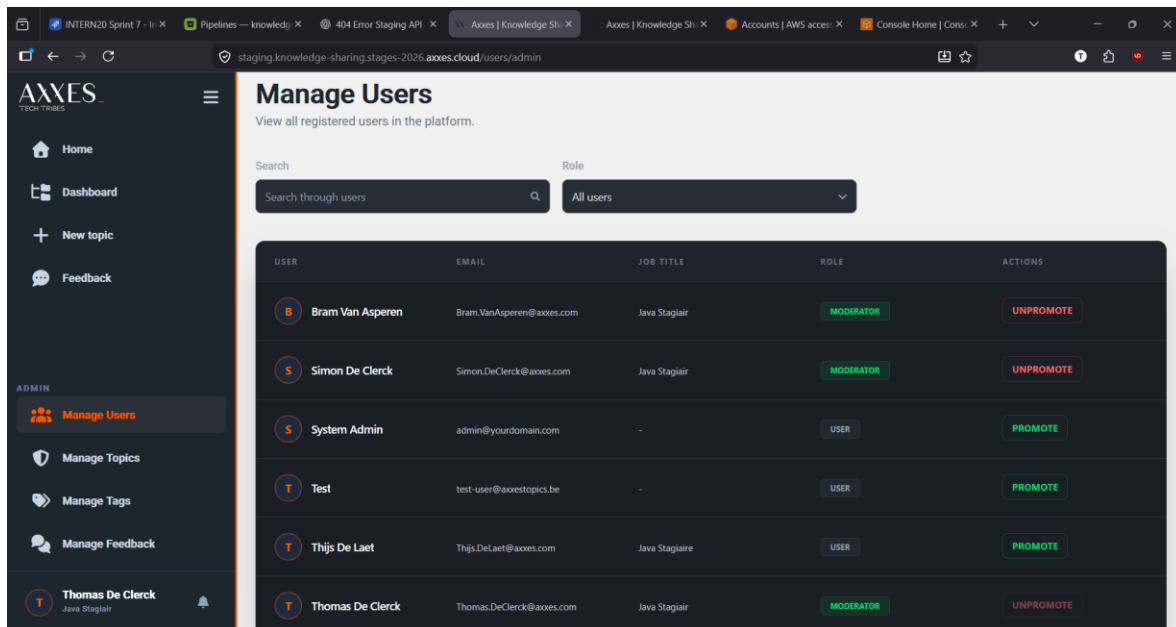
Figuur 35 en Figuur 36 tonen hoe de view counter wordt bijgewerkt nadat een topic in detail werd geopend. Na het bekijken van een topic wordt het aantal weergaven verhoogd en opnieuw weergegeven binnen de applicatie.

3.3.13. Userbeheer CRUD met Azure Entra ID

Een ander onderdeel binnen de realisatie was het gebruikersbeheer van de applicatie. Hiervoor werd gebruikgemaakt van Azure Entra ID. Mijn bijdrage bestond uit het uitwerken van een gebruikersoverzicht en het beheren van rollen, zoals het toekennen van moderatorrechten aan een gebruiker en het opnieuw intrekken van deze rol. Ook het aanvragen en correct verwerken van de nodige rechten en permissies maakte deel uit van dit onderdeel.



Figuur 37: Gebruikersoverzicht binnen de applicatie



Figuur 38: Beheer van moderatorrechten voor een gebruiker

Figuur 37 en Figuur 38 tonen het gebruikersbeheer binnen de applicatie. Via dit overzicht kunnen gebruikers geraadpleegd worden en kunnen moderatorrechten worden toegekend of opnieuw ingetrokken.

3.3.14. Aanvullende bijdragen

Naast de eerder beschreven onderdelen werkte ik ook mee aan enkele grotere technische refactors binnen het project. Deze onderdelen waren niet altijd volledig individueel, maar vormden wel belangrijke bijdragen aan de kwaliteit, structuur en onderhoudbaarheid van de applicatie.

Backend refactor naar DDD en hexagonale architectuur

Een belangrijke aanvullende bijdrage was de refactor van de backend naar een hybride structuur op basis van Domain-Driven Design en hexagonale architectuur. Dit was een ingrijpende refactor waarbij de bestaande backend structuur grondig werd herwerkt. Het doel hiervan was om de applicatie beter op te delen in duidelijke lagen, waarbij de domeinlogica beter gescheiden werd van technische implementatiedetails zoals controllers, repositories en externe afhankelijkheden.

Aan deze refactor werkte ik samen met Simon, Bram en Thijs. Sommige onderdelen werden via pair programming uitgewerkt, terwijl andere delen zelfstandig werden aangepakt. Door deze refactor werd de backend overzichtelijker, beter testbaar en eenvoudiger uitbreidbaar voor toekomstige functionaliteiten.

Backend test refactor

Daarnaast werkte ik mee aan de refactor van de backendtests. Naar aanleiding van feedback van Tim Schmitte, een Axxes collega, en Lieven werd duidelijk dat de bestaande tests te veel gebruikmaakten van mocks en dat er te vaak `@Transactional` werd toegepast.

Mijn bijdrage bestond erin om de ongeveer driehonderd bestaande tests te herstellen en mee te verbeteren, zodat ze beter aansloten bij de gewenste testaanpak. Het overmatig gebruik van mocks zorgde ervoor dat sommige tests te ver afstonden van de echte werking van de applicatie. Ook `@Transactional` werd te vaak gebruikt, waardoor testdata automatisch werd teruggedraaid en bepaalde problemen minder snel zichtbaar werden. Door de tests te herwerken, werd de betrouwbaarheid van de testomgeving verbeterd en sloten de tests beter aan bij realistische scenario's.

Frontend mappenstructuur en facade pattern

Ook binnen de frontend was er nood aan herstructurering. In het begin van het project was de mappenstructuur nog niet volledig consistent, waardoor bestanden en functionaliteiten minder overzichtelijk gegroepeerd waren. Samen met Simon werkte ik mee aan het herstructureren van de frontend, zodat de applicatie duidelijker werd opgebouwd per feature en component.

Daarnaast werd het facade pattern correcter toegepast. Dit patroon helpt om componenten eenvoudiger te houden door logica rond data, state en communicatie met services te bundelen in een aparte facade. Hierdoor werden componenten minder zwaar, werd de code beter leesbaar en werd het eenvoudiger om functionaliteiten verder uit te breiden of aan te passen.

Deze refactor zorgde ervoor dat de frontend beter georganiseerd werd en dat toekomstige ontwikkeling binnen het project vlotter kon verlopen.

Bug fixing en fronten optimalisatie

Tijdens de laatste weken van mijn stage heb ik verschillende bug rapporten van Lieven opgelost. Omdat Bram en Simon nog bezig waren met nieuwe functionaliteiten, focuste ik vooral op bugfixes en kleine fronten verbeteringen. Het grootste onderdeel hiervan was de refactor van de dashboardpagina, waardoor deze overzichtelijker en beter onderhoudbaar werd.

Base facade refactor

Ook heb ik de base facade abstracter gemaakt. Deze werd door meerdere facades gebruikt, maar was nog te sterk gekoppeld aan één specifieke state. Door dit te refactoren werd de facade herbruikbaar en flexibeler voor verschillende onderdelen van de applicatie.

4. Besluit

Dit realisatiedocument beschreef de analyse, technische keuzes en concrete uitwerking van het kennisdelingsplatform voor Axxes. Vanuit de probleemstelling dat kennis binnen de organisatie verspreid zit over verschillende tools en kanalen, werd onderzocht hoe een centrale en gebruiksvriendelijke applicatie deze kennis beter kan verzamelen, structureren en toegankelijk maken.

Tijdens de analysefase werd duidelijk dat bestaande oplossingen zoals SharePoint, Notion en Confluence niet volledig aansloten bij de specifieke noden van Axxes. Daarom werd gekozen voor een eigen oplossing op maat. Via de requirementsanalyse, user personas, het use case diagram, het ERD, de MoSCoW-analyse en de prototypes werd de functionele en technische basis van de applicatie vastgelegd. Deze voorbereiding zorgde ervoor dat de ontwikkeling kon vertrekken vanuit een duidelijke scope en een onderbouwde projectstructuur.

Op technisch vlak werd gekozen voor een combinatie van Angular in de frontend, een Java REST API in de backend, Azure Entra ID voor authenticatie, MySQL als databank en AWS voor de cloudomgeving. Deze keuzes werden niet alleen gemaakt op basis van technische sterktes, maar ook op basis van haalbaarheid, ervaring binnen het team en aansluiting bij de voorkeuren en werking van Axxes. Tijdens de realisatie werd daarnaast veel aandacht besteed aan onderhoudbaarheid en uitbreidbaarheid, onder andere door het gebruik van een feature-based frontendstructuur, het facade-patroon, Angular signals en een backend opbouw volgens DDD en hexagonale architectuur.

Binnen de realisatie werd een werkende applicatie opgebouwd waarin gebruikers topics kunnen aanmaken, bekijken, beheren en ermee kunnen interageren. Afhankelijk van de individuele bijdrage werden verschillende onderdelen verder uitgewerkt, zoals topicbeheer, interactiefunctionaliteiten, AI-ondersteuning, gebruikersbeheer, mediafunctionaliteiten, collaboratief bewerken, comments, ratings, dashboards en responsiviteit. Deze onderdelen dragen samen bij aan een platform dat kennisdeling binnen Axxes eenvoudiger, overzichtelijker en interactiever maakt.

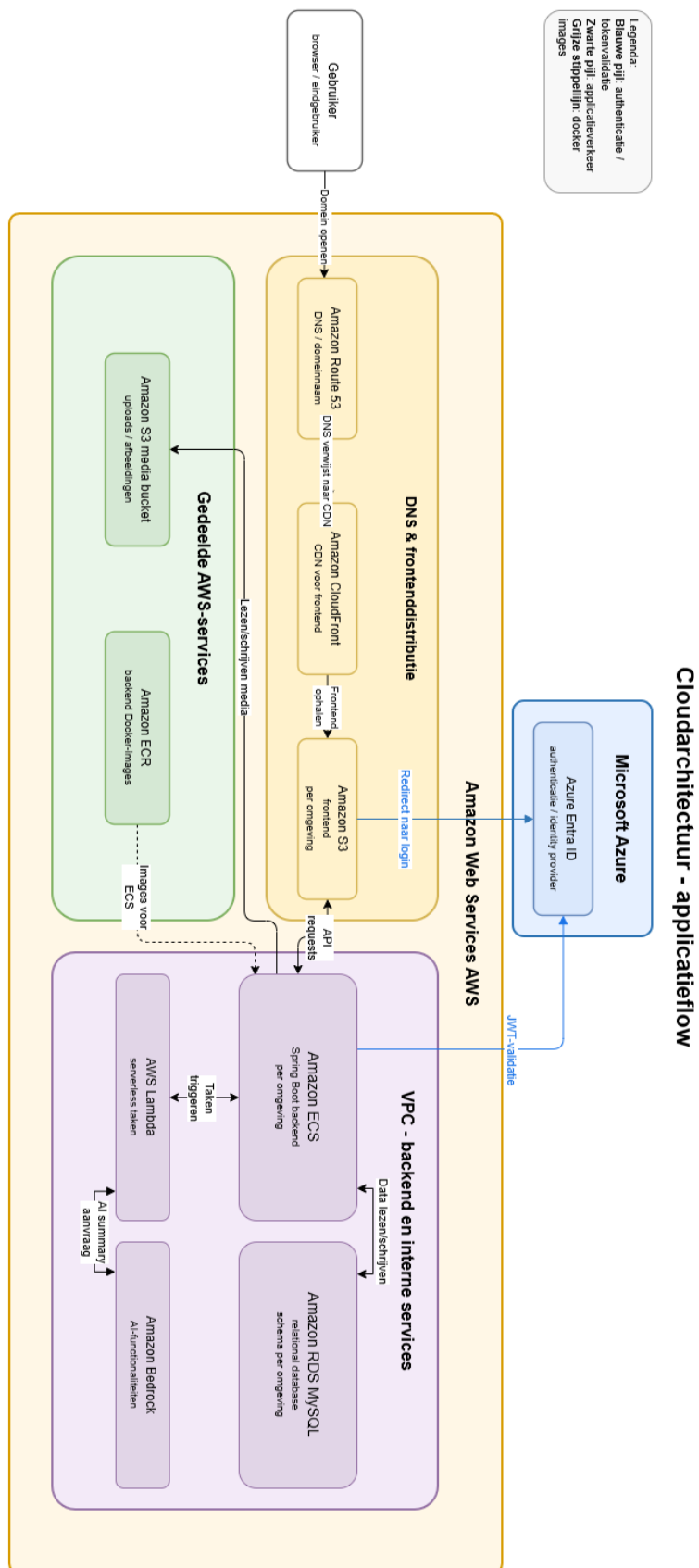
Wanneer wordt teruggekoppeld naar de doelstellingen uit het projectplan, kan gesteld worden dat de belangrijkste doelen grotendeels gerealiseerd werden. Er werd een interne webapplicatie ontwikkeld waarin kennis centraal beheerd en gedeeld kan worden. De applicatie ondersteunt verschillende soorten content, biedt mogelijkheden tot interactie en voorziet beheerfunctionaliteiten om de kwaliteit en structuur van de gedeelde kennis te bewaken. Niet alle mogelijke uitbreidingen werden binnen de scope van de stage volledig uitgewerkt, maar de basis van het platform is voldoende stevig om verder op voort te bouwen.

Voor de toekomst zijn er nog verschillende uitbreidingsmogelijkheden. Zo kan het platform verder verfijnd worden met uitgebreidere statistieken, extra personalisatie, verbeterde zoekfunctionaliteiten, een sterker governanceproces of bijkomende integraties met andere interne systemen. Ook verdere optimalisatie van de gebruikerservaring en bijkomende feedback van Axxes-medewerkers kunnen helpen om het platform nog beter af te stemmen op de dagelijkse werking binnen de organisatie.

Algemeen kan besloten worden dat dit project niet alleen resulteerde in een bruikbare technische oplossing, maar ook in een leerzaam ontwikkeltraject. Doorheen de stage werd ervaring opgedaan met analyse, technologiekeuzes, frontend- en backendontwikkeling, architectuur, samenwerking, testing, documentatie en feedbackverwerking. Het kennisdelingsplatform vormt daardoor zowel een waardevolle bijdrage voor Axxes als een belangrijk onderdeel van de professionele groei tijdens de stage.

LITERATUURLIJST

- Ahmad, K. (2025, December 31). *Angular vs React vs Vue: Which Should You Learn in 2026?* Retrieved February 23, 2026, from dev.to: <https://dev.to/kafeel-ahmad/angular-vs-react-vs-vue-which-should-you-learn-in-2026-1ocg>
- Bloomfire. (2025, March 11). *The Disadvantages of SharePoint: Is It a Fitting Choice in 2025?* Retrieved February 25, 2026, from Bloomfire.com: <https://bloomfire.com/resources/disadvantages-of-sharepoint/>
- El khirani, S. (2024, November 23). *The Best Database Servers: Oracle, SQL Server, MySQL, and PostgreSQL Compared.* Retrieved February 24, 2026, from Medium.com: <https://medium.com/data-science-clarity/the-best-database-servers-oracle-sql-server-mysql-and-postgresql-compared-fb2f4be17360>
- G2.com. (n.d.). *Compare Auth0, Microsoft Entra ID, and Amazon Cognito.* Retrieved February 24, 2026, from G2.com: <https://www.g2.com/compare/auth0-vs-microsoft-entra-id-vs-amazon-cognito>
- Gartner, Inc. and/or its affiliates. (2025, January 21). *Oracle Named a Leader in the 2024 Gartner® Magic Quadrant™ for Cloud Database Management Systems.* Retrieved February 24, 2026, from Oracle.com: <https://www.oracle.com/europe/news/announcement/oracle-named-leader-in-gartner-magic-quadrant-for-cloud-database-management-systems-2025-01-21/>
- Gartner, Inc. and/or its affiliates. (n.d.). *Access Management Reviews and Ratings.* Retrieved February 24, 2026, from Gartner.com: <https://www.gartner.com/reviews/market/access-management>
- Heath, C. (2025, July 28). *Pros and Cons of SharePoint Knowledge Base Software.* Retrieved February 25, 2026, from Document360.com: <https://document360.com/blog/sharepoint-knowledge-base-software/>
- Mika, A., & Vasylenko, J. (2026, January 20). *Angular vs React vs Vue.* Retrieved February 23, 2026, from ramotion.com: <https://www.ramotion.com/blog/angular-vs-react-vs-vue/>
- Miller, D. (2024, March 3). *Notion is NOT the king of Knowledge Management Tools, here's why.* Retrieved February 25, 2026, from Polyinnovator.space: <https://www.polyinnovator.space/notion-is-not-the-king-of-knowledge-management-tools-heres-why/>
- Redgate Software. (2026, February). *DB-Engines Ranking.* Retrieved February 24, 2026, from DB-Engines.com: <https://db-engines.com/en/ranking/>
- Řeřicha, J. (2025, October 14). *Knowledge base alternatives: Is Confluence the best choice for you?* Retrieved February 25, 2026, from Easyredmine.com: <https://www.easyredmine.com/news/knowledge-base-alternatives-is-confluence-the-best-choice-for-you>
- Roach, J. (2024, September 20). *PostgreSQL vs. MySQL: Choosing the Right Database for Your Project.* Retrieved February 26, 2026, from Datacamp.com: https://www.datacamp.com/blog/postgresql-vs-mysql?utm_cid=23340058068&utm_aid=192632749129&utm_campaign=230119_1-ps-dscia%7Edsa-tofu%7Esql_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9197437-&utm_mtd=-c&utm_kw=&utm_source=google
- Shah, V. (2026, February 17). *Angular Vs React Vs Vue: Which One To Choose.* Retrieved February 23, 2026, from tatvasoft.com: <https://www.tatvasoft.com/blog/angular-vs-react-vs-vue/>
- Subramanian, D. (2026, January 09). *Top 10 API Frameworks: Choose Your Best Fit for 2026.* Retrieved February 24, 2026, from Digitalapi.ai: <https://www.digitalapi.ai/blogs/top-10-api-frameworks-choose-your-best-fit-for-2026>
- Tech Formation. (2025). *React vs. Angular vs. Vue.js: Choosing the Best JavaScript Framework for Your Project.* Retrieved February 23, 2026, from Techformation.io: <https://www.techformation.io/blog/react-vs-angular-vs-vue-js>



Figuur 39: Overzicht van de cloudinfrastructuur